科研文献助手

基于混合检索与结构化抽取的论文阅读系统

技 术 文 档

项目名称	科研小助手 Agent
作品类型	AI 应用 / 检索增强生成
代码仓库	https://github.com/CSGE4/research-paper-assistant
在线演示	https://research-paper-assistant-yx.streamlit.app/
演示账号	无需账号，打开即可使用
文档版本	v2.0
编写日期	2026 年 11 月

目 录

第 1 章 项目概述与需求分析 4
1.1 项目概述：问题的来源 4
1.2 现有方案为什么不够 4
1.3 需求分析与验收标准 4
1.4 使用场景与边界 4
第 2 章 系统架构设计 6
2.1 两条流水线 6
2.2 分层与模块 6
2.3 AI 技术选型与理由 6
2.4 数据形态流转 7
2.5 三条关键设计决策 8
第 3 章 核心功能实现与 AI 技术方案 9
3.1 总体架构与调用链 9
3.2 入库流水线 9
3.3 双通道检索与分数融合 10
3.4 AI 集成：Agent 编排与工具设计 11
3.5 AI 约束：提示词与防编造 12
3.6 存储设计与一致性 13
第 4 章 工程实现与质量保障 15
4.1 入库事务与幂等设计 15
4.2 AI 优化策略：延迟与成本 16
4.3 缺陷排查记录 17
4.4 部署与配置 19
4.5 代码规模 20
第 5 章 测试与部署 22
5.1 测试方法与评测口径 22
5.2 结构化抽取结果 22
5.3 检索对比实验 22
5.4 融合机制分析 23
5.5 工具与编排实验 23
5.6 端到端性能与成本 24
第 6 章 技术创新点与应用创新点 25
6.1 检索层：两路召回的可比性处理 25
6.2 生成层：上下文归属与防编造 25
6.3 编排层：先看清单再检索 26
6.4 成本层：把「思考」当成可开关的开销 26
6.5 应用创新点 26
6.6 与同类方案的差异 27
第 7 章 总结与展望 28
7.1 检索层面 28
7.2 解析层面 28
7.3 工程层面 29
7.4 评测层面 29
7.5 后续改进 29
7.6 总结 30

第 1 章 项目概述与需求分析

1.1 项目概述：问题的来源

起点是一个具体的阅读场景，不是"给论文加一个聊天框"。读英文论文的阻力集中在三处：

阻力	具体表现	后果
语言	长句嵌套、术语密集（ablation、inductive bias、zero-shot transfer），一句套三个从句	一页读十分钟，读到第三节放弃
结构	方法在正文中段，超参设置常放进附录，实验细节散在图表说明里	想确认某个超参取多少，要来回翻全文
回找	当时看懂了，几天后写综述或复现时记不住在哪一页	引用不了，等于白读

以 Attention Is All You Need 为例：摘要读得懂，模型结构那一节读不动；过几天要确认某个超参写在哪，又从头翻一遍。

由此得到核心判断：

需求不是"替我把论文读完"，而是"让我能在一分钟内核对我读到的那段结论"。

前者是黑盒；后者可验证——拿到一句中文结论和它的出处，翻到那一页自己确认。回答必须带文件名与页码、模型不许用先验知识补细节，都由此而来。

1.2 现有方案为什么不够

把论文交给现成工具，有三条路：

方案	能做到	缺口
通用对话 AI	概括准确、响应快	答案来自训练知识而非这篇论文，出处靠编；同一问题两次答案可能不同
通用"论文 RAG"助手	能检索原文并回答	多只处理文本块，作者、表格、文献、附录拿不到；页码不可信；没有按节翻译
传统翻译工具	逐段机翻	术语按字面翻，公式与表格排版崩坏；译文不标页码，要两份材料来回切

问"Transformer 是哪一年提出的"，通用对话 AI 会直接答"2017 年"——答案没错，但来自模型参数而不是这篇论文。本系统要求只依据库中原文作答，原文没写就说"库中没有明确说明"，并禁止从文件名 `1706.03762` 推断年份：区别不在对错，而在能不能被核对。

1.3 需求分析与验收标准

每条需求都要有可执行的验收口径，否则"做完了"只是自我感觉。下表数字全部来自可复现的评测配置。

需求	验收口径	实测 / 现状
检索能命中	37 题上 hit@10 ≥ 30/37，hit@1 ≥ 15/37	hit@10 **33/37**、hit@1 **17/37**、MRR **0.585**（α=0.20，Top-10）
检索足够快	单次混合检索均值 ≤ 1 秒	均值 **0.50 秒**，最大 **0.96 秒**
端到端可接受	本地 ≤ 35 秒，云端 ≤ 40 秒	本地 **32 秒**，云端 **34 秒**；按节翻译一次 **19 秒**
拒答可信	库里没有的一律拒答，库里有的不许误判为没有	独立拒答题集**未统计**；多篇覆盖任务上，无目录工具时 6 篇误报 3 篇"库中没有"，接目录工具后 6 篇全覆盖
成本可控	单次问答输出 token ≤ 500，思考 token 为 0	输出 **464** token，思考 **0**，正文 **948 字**（上下文 **8888** 字）；单价**未统计**
结论可核对	来源由检索结果计算，不从模型回答里解析	`rag.py` 返回 `{文件名: {页码}}`，模型只负责复述

前两条是贯穿全系统的硬约束，不是界面效果：它约束检索结果的组织方式（每块带 `[文件名 第 N 页]` 前缀）、提示词写法（只许复述原文，页码只能取自工具返回），也约束工具设计（`local_search` 必须把文件名、页码和正文一起返回）。延迟取 32 秒与 34 秒而不是"几秒"，是因为链路上有一次 8888 字上下文的生成，单次生成实测就 8.77 秒。

拒答有两侧：该拒的不拒（编答案），和不该拒的拒了。后者可量化——同一道"这几篇分别用了什么方法"，不接目录工具时 6 篇只覆盖 2 篇、3 篇被误报为"库中没有"，而它们都在库里；接 `list_documents` 后 6 篇全覆盖。可见拒答错误的主因不是模型不听话，而是它不知道库里有什么。

1.4 使用场景与边界

目标用户：需要读英文论文但效率不高的本科生与低年级研究生。不是资深研究者——后者要的是文献管理、引用网络与批量分析。

典型场景：初读时问核心贡献与已有方法的差别；核对细节时问具体数字或条件；读不懂时按节要译文；组会前先扫一遍库里有哪几篇、各自用了什么方法。前两种由检索与生成承担，第三种靠按节翻译，最后一种先列目录再逐篇检索。

明确不做：论文写作、综述自动生成、跨论文立场对比——这三类更容易产出无法核对的结论。

边界是使用前提，不是待办事项：

- 语料规模：当前 6 篇、183 页、727 块；上到几千块后，BM25 每次入库全量重建会成为瓶颈。
- 表格与断字：表格块 markdown 化后与"超参数"这类字面提问重合度低，有时要第二次检索才命中；第 1 页正文常被解析成 `**Lo** w- **R**ank`，界面显示为「Lo w-Rank」。
- 联网：未配 `TAVILY_API_KEY` 时开关不可用，工具返回一句提示而不是让整页报错。
- 覆盖：跨文档对比取决于检索覆盖，只召回一篇时模型只能给出一边。
- 一致性：六份存储非原子写入，云端重启可能部分回滚（见 2.5）。

第 2 章 系统架构设计

2.1 两条流水线

系统由两条独立流水线构成，只在向量库与结构化文件处交汇。

```
【入库流水线】每篇执行一次，产物落盘
PDF
 ├─ pymupdf4llm(page_chunks=True) ──> 逐页 Markdown（183 页，页码锚点在此建立）
 │      ├─ 结构化抽取 ──> chroma_structured.json
 │      │                  （102 表 / 138 图 / 546 条参考文献 / 84 附录 / 78 作者）
 │      └─ SemanticChunker → 合并碎块 → 递归切分(1500/150)
 │            ├─ text-embedding-v4 ──> chroma/（727 块）
 │            └─ jieba 分词 ──> bm25.pkl
 └─ 页级向量 ──> embedding.json（重复上传查重）

【查询流水线】每次提问执行，不写盘
问题
 ├─ 向量召回 Top-10（cos = 1 - d²/2）
 ├─ BM25 召回 Top-10（k1=1.5, b=0.75）
 └─ 融合 0.20·BM25_norm + 0.80·cos ──> Top-10 块（带文件名与页码）
          └─ 拼上下文（8888 字）──> LLM 关思考 ──> 回答（8.77 秒）
外层：Agent 选工具、控检索次数（≤6）、按 thread_id 存记忆
```

入库是重操作（解析、抽取、逐块嵌入），单篇耗时未统计；查询是轻操作，检索 0.50 秒、生成 8.77 秒，且不碰 PDF、不重算结构化字段。`page_chunks=True` 是设计的支点：解析结果因此变成"每页一个 dict"，页码才能跟着每个块进入向量库元数据。

2.2 分层与模块

层	模块	职责
展示层	`web.py`	上传、入库进度、相似文档三选一、结构化展示
展示层	`pages/app.py`	问答页：流式输出、进度文案、检索次数与耗时
应用层	`agent.py`	一个 agent、四件工具、提示词按开关切换
应用层	`tool_3.py` / `tool_1.py` / `tool_2.py` / `tool_4.py`	本地检索 / 按节翻译 / 联网 / 列目录
应用层	`search_budget.py`	检索次数计数与额度判断，两个检索工具共用
应用层	`rag.py`	检索 → 拼上下文 → 作答 → 返回来源
检索层	`vctor.py`	双路召回：向量 + BM25
检索层	`bm25_score.py`	分数归一化与加权融合
数据层	`knolege.py`	入库主流程、六份存储写入、增删
数据层	`paper_extractor.py`	规则抽取十类结构化字段
数据层	`model_extractor.py`	规则失败时的模型兜底，带原文子串校验
数据层	`md5_saved.py` / `fs.py`	文件指纹登记 / 读写工具
配置	`config.py` / `glossary.py`	路径与参数 / 翻译术语表

代码规模：21 个 `.py`、4577 行，其中代码 3429 行、注释 667 行、空行 481 行。依赖方向单向：展示层 → 应用层 → 检索层 → 数据层；数据层不感知上层，所以入库脚本可脱离界面运行，检索层也能单独评测（1.3 的 37 题只跑 `vctor.py`）。

一处代价要指明：`vctor.py` 在 import 时构造全局 `Knolege`，`vctor_story_search` 又自建一个 `Chroma` 客户端，指向同一持久化目录——功能没问题，但连接与内存都是双份。

2.3 AI 技术选型与理由

环节	选择	主要备选	理由
PDF 解析	`pymupdf4llm`	PyPDF2、pdfplumber	直接输出保留标题层级与表格的 Markdown，支持按页返回
结构化抽取	自研规则 + 模型兜底	全交给模型	十类字段里九类格式稳定，规则零成本可复现
切块	`SemanticChunker` + 合并 + 递归兜底	固定长度递归	固定长度会把句子切在两块里；语义切块不保证长度上限
向量模型	`text-embedding-v4`	本地 bge、m3e	中英跨语言匹配是硬需求（中文提问、英文原文），无需 GPU
向量库	Chroma	FAISS、Milvus	元数据过滤（按 `source`、`page_number` 取页码），本地持久化，零运维；FAISS 无过滤要另存页码表，Milvus 需独立部署
关键词召回	`rank_bm25` + jieba	只用向量	纯向量对专有名词、数字、缩写不敏感

环节	选择	主要备选	理由
问答编排	LangGraph 的 `create_agent`	手写调度、子图	多轮工具循环、记忆、换提示词都有现成接口，不必自维护循环终止
会话记忆	`SqliteSaver`	内存字典、Redis	按 `thread_id` 落盘，重启不丢；Redis 多一个服务，与零运维冲突
界面	Streamlit 多页	FastAPI + Vue	优先保证功能完整度；代价是脚本每次交互重跑，状态只能挂 `session_state`

为什么是 qwen3.8-flash 且关掉思考。 这是影响最大的一处选择。同一段输入（同一次检索结果，上下文 8888 字）三种配置实测：

配置	耗时	输出 token	其中思考	正文
flash 默认（开思考）	82.27 s	4410	4103	678 字
flash 关思考	**8.77 s**	**464**	**0**	**948 字**
max 默认（开思考）	41.81 s	1610	1604	7 字（只回了「参考资料未提及」）

输出 token 九成以上花在思考上：82.27 秒里 4410 个 token 有 4103 个是思考，正文只占 678 字。这条链要的是照原文复述，不需要推演，思考是纯开销；关掉后不是变糙，而是更快更长——8.77 秒、464 token 写出 948 字。降档位也不如关思考：max 默认 41.81 秒、1610 token 里 1604 是思考，正文只有 7 字，预算全烧在自我推演上还答不出来。

端到端：同一问题（"attention is all you need 中，超参数是多少"）79.29 秒 → 32 秒（本地）；云端 34 秒，翻译一次 19 秒。设置上同时钉死 `timeout=45, max_retries=0`——重试会把一次卡住放大成两三倍，而一次问答要调两三次模型。

```
model = ChatOpenAI(
    model="qwen3.8-flash",
    timeout=45, max_retries=0,
    extra_body={"enable_thinking": False},   # 思考占输出 token 八成以上，关掉
)
```

提示词切换靠中间件完成，不需要建两个 agent：

```
main_agent = create_agent(
    model,
    tools=[local_search, translate_paper, web_search, list_documents],
    middleware=[pick_prompt],          # 读 runtime.context 决定是否追加联网规则
    checkpointer=checkpoint,
)
```

`pick_prompt` 是 `@dynamic_prompt` 中间件，读 `runtime.context["use_web"]`，联网时返回 `main_prompt + web_prompt`，否则只返回 `main_prompt`（两套文案单独放在 `promot.py`，改提示词不必翻逻辑）；同一个对象因此能同时服务联网与不联网两种会话。

2.4 数据形态流转

一份 PDF 进入系统后被拆成六种落盘形态，各有用途，写入顺序固定：

#	形态	产物	规模（6 篇实测）	用途
1	向量块	`chroma/`（collection `knolegy`）	727 块	检索与问答的最小单位
2	摘要	`chroma_summary.json`	6 条，每篇前 2000 字	相似判断时给模型看的材料
3	结构化字段	`chroma_structured.json`	102 表、138 图、546 条文献、84 附录、78 作者	详情页展示、列目录工具
4	分词索引	`bm25.pkl`	727 条	关键词召回
5	指纹	`md5.txt`	6 条	拦截完全重复的上传
6	页级向量	`embedding.json`	183 页	判断是否已有相似文档

块类型：正文块 645、表格块 82，合计 727。

```
self.chroma.add_texts(all_chunks, metadata_list, ids)   # 1 向量库
...                                                      # 2 摘要  3 结构化
self.build_bm25(all_chunks, filename, page_nums)         # 4 分词索引
save_md5(str_hashlib, filename)                          # 5 指纹
self.embeddings_dict[filename] = page_embedding          # 6 页级向量
```

顺序是刻意的：把"这篇已入库"的登记（md5、页级向量）放最后，中途任何一步失败都不留半条记录；否则会出现 md5 拦不住重传、`embedding.json` 却认得"自己"的幽灵记录。

六种形态互不冗余：块用于检索，页级向量用于查重，结构化字段用于展示与目录，逐页原文用于抽取与翻译。块 id 是确定性的 {文件名}_p{页码}_{块号}，重传按 id 覆盖而不插入第二份；代价是重传后块数变少时旧 id 残留，所以同名覆盖必须先删后写。

2.5 三条关键设计决策

决策一：检索层与生成层分离。 rag.py 只做一件事：保证模型看到的是库里的原文，而不是它自己的常识。它不选工具、不维护记忆、不规划多轮——那些在 agent.py；它也不挂 LangChain 的 retriever 接口，因为该接口只认单一检索器，塞不进两路融合。分离让评测能分层：37 题只跑检索层、不启动 agent，几十秒就能比完 α、候选集 K、归一化方式。

来源同样由这一层决定：

```
sources = self.collection_source(result)   # 从检索结果聚合，不信模型写的页码
```

模型写的页码不可信——它可能只用了 10 块里的 2 块，却把 10 块页码全写上；也可能自己推算，看起来精确其实是编的。

决策二：用六份存储换查询速度。 入库写六个地方，查询时每一路直接读现成产物：检索读 chroma/ 与 bm25.pkl，查重读 embedding.json，展示与目录读 chroma_structured.json，拦截重传读 md5.txt。只留一份源数据的话，每次查询都要重新解析 PDF 或重算嵌入，检索 0.50 秒这个量级不可能达到。

代价必须写明：六份存储是**非原子写入**，没有事务包裹。中途失败或云端实例重启可能只写进去一部分——向量库有这篇、chroma_structured.json 没有，表现为"文件列表显示 7 个文件，却不报相似"。缓解只有两条：把"已入库"的登记放最后；删除时逐文件清理。这不是完备方案，是当前规模下接受的折中。

决策三：提示词即接口。 工具与模型之间不靠结构化 schema 传结果，而靠一行固定格式的字符串。

```
return "%s\n\n 检索文件来源以及页码：%s" % (result["answer"], "；".join(sources))
```

agent.py 的提示词要求模型原样保留这一行，页码只能取自工具返回，没有引用原文时整行不写。这条约定被三处同时依赖：提示词的格式要求、tool_3.py 的拼装、页面上的展示；改一处而不同步改另两处，规则立即失效。好处是不必为每个工具定义输出 schema；代价是契约隐式，活在字符串里，没有类型检查。

同一思路也用在上下文前缀上，[文件名 第 N 页] 同样是接口：

```
context = "\n\n".join(
    "[%s 第%s 页] %s" % (i["metadata"]["source"], i["metadata"]["page_number"], i["text"])
    for i in result)
```

文件名不能省。10 个块可能来自 6 篇文献，只给页码的话模型分不清这句话是谁说的——它会把 CLIP 相关工作里提到的 Transformer 说成 Transformer 自己的做法，也会把《Scaling Laws》的 batch size 当成 Transformer 的超参数。补上文件名后，同一份检索结果，模型自己就把两篇的设置分开了。

第 3 章 核心功能实现与 AI 技术方案

第 2 章给了分层与模块划分，本章落到可核对的实现：每一步调用了什么、参数写在哪、为什么取这个值。代码规模 21 个 .py 文件、4577 行（代码 3429 行、注释 667 行、空行 481 行）；语料 6 篇论文、183 页、727 个文本块。数字均取自实测记录，未统计的量明确标注。

3.1 总体架构与调用链

系统只有两条主干：写链把 PDF 变成可检索的数据，读链把问题变成带出处的答案。

链	入口	落点
写（入库）	`Knolege.upload_str_pdf()`，knolege.py	六份存储
读（问答）	`main_agent.invoke(..., context={"use_web": …})`，agent.py	答案正文 + 来源页

写链的六个阶段：

阶段	动作	说明
① 查重	`get_str_md5` → `check_md5`	内容完全相同则直接返回，不解析、不嵌入
② 解析	`pymupdf4llm.to_markdown(page_chunks=True)`	每页一个 dict，页码从这一步起被保留
③ 页级嵌入	`embeddings.embed_documents(page_texts)`	每页一条向量，既用于查相似，也是这篇的指纹
④ 暂存	`temp_pdf[filename] = {...}`	等用户决定覆盖/都留，此时库里没有任何写入
⑤ 相似判定	页向量取均值 → 逐篇余弦	超阈值则交模型二次判断，返回 JSON 建议
⑥ 写库	`_save_to_all_db`	六份存储依次写入，顺序见 3.6

读链的调用顺序：

```
pages/app.py 提问
└─ main_agent.invoke(..., context={"use_web": bool})
   ├─ 中间件 pick_prompt：按 context 拼系统提示词
   ├─ 模型调用 #1：只决定调哪个工具，不写答案
   ├─ local_search(query) → Rag.get_answer
   │    ├─ result_search：向量 + BM25 两路召回，融合取 Top-10
   │    ├─ 拼上下文：每块前缀 [文件名 第 N 页]
   │    └─ 模型调用 #2：照原文作答
   └─ 模型调用 #3：把工具返回组织成最终回复
```

一次提问至少两次模型调用：第一次开口时检索结果还不存在，模型不可能在没读到原文的情况下作答，所以「选工具」与「写答案」必须分开。

导入期就在执行的代码是最易被误读的一层。vctor.py 在模块级写了 `global_knowledge = Knolege()`，import 即执行：读 bm25.pkl、建嵌入客户端、打开 ./chroma。因此 web.py 里加载 st.secrets 的几行必须排在所有 import 之前，晚一行 `Knolege()` 就会拿着空 key 初始化完成。

3.2 入库流水线

3.2.1 六个步骤

步	操作	参数
1	`pymupdf4llm.to_markdown` 转 Markdown	`page_chunks=True`，按页产出
2	`SemanticChunker` 语义切分	按百分位找断点，切在相邻句子嵌入最不相似处
3	`_merge_small_chunks` 合并碎块	相邻块合并到最小长度，超短块丢弃
4	`RecursiveCharacterTextSplitter` 二次切分	`chunk_size=1500`、`chunk_overlap=150`
5	`text-embedding-v4` 向量化	与切块器共用同一个嵌入模型
6	写入 Chroma	`collection_name="knolegy"`、`persist_directory="./chroma"`

三档切块各解决一个问题：

档	解决什么	不这么做的后果
一档语义切分	在「意思变了」处断开，而非固定字数	固定长度会把一个论证切成两半，两块都不完整
二档合并	语义切分会产出大量碎块；页码、页眉页脚成了独立短块	内容就是一个数字的页码块会直接进入检索结果
三档长度兜底	语义切分只保证边界，不保证长度上限	松散论文可能切出超长块，占满嵌入模型上下文

3.2.2 切出来的规模

指标	数值
论文数	6
总页数	183（逐篇 15 / 30 / 48 / 26 / 34 / 30）

指标	数值
文本块	727
表格块 / 正文块	82 / 645
结构化抽取：表 / 图	102 / 138
结构化抽取：参考文献 / 附录 / 作者	546 / 84 / 78

3.2.3 块 id 是确定性的

```
ids.append(f"{filename}_p{data_no}_{chunk_idx}")
```

同一篇重传时，`add_texts` 按 id 覆盖而不会插入第二份。代价是：若重传后切出的块数变少，多出来的高编号旧块不会被覆盖，会残留在库里。所以「覆盖旧文档」分支必须先 `chroma.delete(where={"source": filename})` 再写入——这个顺序不是洁癖，是上述 id 方案的直接后果。

3.3 双通道检索与分数融合

3.3.1 向量一路：距离必须还原成相似度

Chroma 的 `similarity_search_with_score` 返回的是**平方 L2 距离**，不是相似度。在向量已归一化的前提下有 `cos = 1 − d²/2`：

```
cos_sim = 1 - (d ** 2) / 2
cos_sim = max(min(cos_sim, 1.0), 0.0)     # 浮点误差可能出界
```

不换算就直接加权，等于把「距离越小越好」和「分数越大越好」两种方向相反的度量相加，量纲和单调性都是错的。

3.3.2 BM25 一路：为什么要重算绝对分

`bm25.pkl` 里存的是 `BM25Okapi` 索引，它的 `get_scores` 只能一次给出「整个查询对全部文档」的分数向量，无法单独算某一条候选的绝对分。融合需要的正是绝对分，于是在 `bm25_score.py` 里另写了一个 `BM25Calculator`：

```
for bm_item in bm25_res:
    score_map[text]["bm25"] = bm25.get_score(query, text)   # 真分，不是排名分
```

参数用标准取值 `k1=1.5`、`b=0.75`，没针对本项目调过。索引在每次查询时现建，代价是要把全部 727 块重新分词一遍——这是检索延迟的主要来源之一，实测均值 0.50 秒、最大 0.96 秒。

3.3.3 归一化：sigmoid 优于 min-max 的地方

BM25 原始分无上界、且与语料规模相关，必须压到 `[0,1]` 才能与余弦相似度加权。这里用带绝对锚点的 sigmoid：

```
def bm25_abs_normalize(score, beta=4.0, scale=4.0, raw_threshold=0.5):
    if score < raw_threshold:
        return 0.0
    return 1.0 / (1.0 + math.exp(-(score - beta) / scale))
```

两者的差别不在曲线形状，而在**零点由谁决定**：

维度	min-max	sigmoid 绝对归一化
锚点来源	本次检索的最高分	固定原始分锚点（`beta=4.0`）
整批结果都差时	最高分仍被拉成 1.0	全部趋近 0，融合分随之下沉
「有一篇很好」与「都不好」	数值上无法区分	可以区分
跨查询可比	不可比	可比
对候选池大小的敏感度	敏感（池子越大最高分越可能虚高）	不敏感

min-max 的致命处是它把每一次检索都强行归一：一篇真正相关的论文和一批全不相关的块，经变换后最高分都是 1.0，α 加权就失去筛选作用。sigmoid 给的是绝对刻度——原始分 4.0 附近算「确有相关」，高于它继续上升，低于 `raw_threshold` 直接归零。分数因此跨查询可比，不会因本次检索质量差而被放大。

3.3.4 α 的取值与实验依据

最终分数为：

```
final = α · BM25_norm + (1 − α) · cos
```

`hybrid_search` 的签名里 `alpha=0.20`，实际调用传入 Top-10。α 不是拍的，是在 37 题评测集上扫描出来的：

方法	hit@1	hit@10	MRR
纯向量	15/37	33/37	0.532
纯 BM25	10/37	26/37	0.423
融合 α=0.20	**17/37**	**33/37**	**0.585**
oracle 上界	—	—	0.642

α 扫描结果：

α	0.10	0.20	0.35	≥0.60
MRR	0.603	0.585	0.547	0.485

取 0.20 而不是 MRR 更高的 0.10，理由有三条：

1. **0.10 与 0.20 的差距在噪声量级**。37 题上单题命中变化就能带来同量级抖动，把 0.603 当真实优势是过拟合。
2. **两路强弱有明确物理解释**。语料是英文、提问是中文，跨语言匹配主要靠向量；BM25 那一路的贡献集中在提问里出现的英文标识符（`r=4`、`CIFAR-10` 这类）。这决定了关键词权重应当压低，而不是对半分。
3. **α 过高会明显劣化**。到 0.60 以上 MRR 掉到 0.485，已低于纯向量；0.20 距离这条悬崖足够远。

融合把 hit@1 从纯向量的 15/37 提到 17/37，hit@10 与纯向量持平（33/37）——改善的是**排序**，不是**召回**。这与 oracle 上界 0.642 的差距也吻合：提升空间在排序与覆盖。

3.3.5 两个已知的实现限制

其一，候选池大小不是瓶颈。 K 取 10、25、50、100，MRR 都是 0.550。扩大召回没有收益，漏答的原因不在池子太小，而在排序。

其二，关键词分只在 BM25 自己召回的那 10 条里计算。 融合只遍历 `bm25_res`，只被向量召回的块其 `bm25` 保持默认值 `0.0`，加权时相当于「关键词完全不匹配」。两路 Top-10 的重叠块数**未统计**。把全部候选的 BM25 分补齐后效果反而下降，因此当前保留此实现，结构性改进列入后续工作。

3.4 AI 集成：Agent 编排与工具设计

3.4.1 一个智能体，四件工具

```
main_agent = create_agent(
    model,
    tools=[local_search, translate_paper, web_search, list_documents],
    middleware=[pick_prompt],
    checkpointer=checkpoint,
)
```

工具	实现文件	何时使用	返回形态	是否调模型
`local_search`	`tool_3.py` → `rag.py`	问内容、问数字、问原因	答案 + 「检索文件来源以及页码：…」	是（检索后生成）
`translate_paper`	`tool_1.py`	用户明确要「译文」	按节中文译文 + 页码	是
`web_search`	`tool_2.py`	联网开关打开且本地查不到	标题 + 链接 + 摘要	否（外部检索）
`list_documents`	`tool_4.py`	不知道库里有什么、要覆盖多篇	「文件名 + 标题 + 页数」清单	否

职责边界靠 **docstring 承担路由**：模型是读工具描述来决定调哪个的，所以 `translate_paper` 的描述里写了「用户问内容、问含义时不要用本工具」，否则模型会拿译文回答内容问题，答案缺结论。`local_search` 用单例 `rag_instance` 缓存 `Rag` 实例——一次问答里可能连调几次，每次新建等于每次重开向量库。

`web_search` 是一层包装而非直接挂 Tavily：未配 `TAVILY_API_KEY` 时返回「联网搜索未开启…只能依据本地检索结果作答」，让模型退回本地检索，而不是让问答页抛异常。

3.4.2 `list_documents`：模型不知道库里有什么

这是四件工具里唯一不调模型、不检索的一件。它直接读 `chroma_structured.json`，按文件名排序输出标题与页数：

```
docs = load_structured()
for name in sorted(docs):
    title = ((info.get("title") or {}).get("value") or "").strip()
```

因为只是一次本地 JSON 读取，**0.05 秒返回**——比一次检索（均值 0.50 秒）快一个数量级，且完全不产生模型调用。

为什么需要它，实验说得最清楚。题目是「这几篇论文分别用了什么方法？各自的创新点是什么？」，要求覆盖全部六篇：

配置	工具调用	总耗时	覆盖	结果
无上限	19 次检索	279.46 s	6 篇	内容全，但过渡语混进答案，且谎称联网查到
上限 6，无清单工具	10 次检索	83.93 s	2 篇	3 篇误报「库中没有」，虽然它们在库里
上限 6 + `list_documents`	1 次列目录 + 6 次检索	125.54 s	**6 篇**	六篇分节，标题全对

三行数据指向三个不同的问题：

· **无上限**证明「多查」不等于「查对」。19 次检索把六篇都扫到了，代价是 279.46 秒，过程叙述污染了答案，还出现「谎称联网查到」这类编造——放开次数会同时放开编造的空间。

· **上限 6 无清单工具**暴露了真正的瓶颈：模型不知道库里有哪几篇，只能从问题里猜关键词。猜偏一次，一次额度就白花；10 次检索只覆盖 2 篇，还把 3 篇在库里的文献报成「库中没有」。这是**元信息缺失**。

· **加上清单工具**后，模型先拿到六篇的文件名与标题，再按标题里的关键词检索，6 次额度覆盖 6 篇。总耗时 125.54 秒高于 83.93 秒，多出来的是真正读完六篇的成本。

适用边界同样清楚。另一道题「Transformer、CLIP、LoRA 三篇的核心方法有什么区别？」，模型已知三篇名字，直接 3 次检索、51.74 秒、1142 字，给出三节加一句关系小结，没有调用清单工具。**问题里已经点名文献时，清单是多余的**；只有当问题说「这几篇」「所有文献」「分别」或模型对库内容一无所知时，它才是必需品。

3.4.3 `pick_prompt`：同一个智能体，两套提示词

「联网/不联网」两套规则必须挂在**同一个**智能体上。`create_agent` 在构造时就把系统提示词编进了图，`invoke` 没有 `system_prompt` 参数，因此改用 `@dynamic_prompt` 中间件，在每次模型调用前替换系统消息：

```
@dynamic_prompt
def pick_prompt(request) -> str:
    use_web = False
    if request.runtime.context:
        use_web = request.runtime.context.get("use_web", False)
    return main_prompt + web_prompt if use_web else main_prompt
```

开关值由页面通过 `invoke(..., context={"use_web": …})` 传入，因此同一个 `main_agent` 能同时服务两种会话，不必建两个智能体、也不必共享两套图。

中间件的执行次数等于模型醒来的次数，即 `1 + 工具调用次数`。它替换的是**本次请求的系统消息**，不写入历史。

`web_prompt` 里有一条容易漏掉的必需项：必须**显式覆盖**前面「本地查不到就回答『库中没有相关资料』」那条。不覆盖的话，开关打开了模型也只会继续回答「库中没有」，联网等于没开。

3.4.4 `SqliteSaver` 按 `thread_id` 存记忆

```
connection = sqlite3.connect(config.sqlite_path, check_same_thread=False)
checkpoint = SqliteSaver(connection)
checkpoint.setup()
```

`check_same_thread=False` 是必需的：Streamlit 脚本会在不同线程里重跑，SQLite 默认禁止跨线程复用连接。`setup()` 负责建表，缺了它第一次 `invoke` 就报表不存在。

记忆的单位是 `thread_id`，由调用方通过 `config=config.config_thread` 传入。同一 `thread_id` 的多轮提问共享消息历史，换一个就是一份全新的会话记忆——「记住上文」与「开新对话」是同一机制的两面。

要诚实说明：`checkpointer` 存的是**消息**，不是检索结果。上一轮检索到的原文不会自动回到上下文，模型若在新一轮引用旧结论，靠的是历史消息里的文本。这也是提示词要求「每条结论都要有当次工具返回的原文支撑」的原因。

3.4.5 模型参数

参数	取值	原因
`model`	`qwen3.8-flash`	同一代低一档，够用于照原文复述
`enable_thinking`	`False`	思考占输出八成以上
`timeout`	45 秒	覆盖正常最慢的一次生成
`max_retries`	0	重试把一次卡住放大成 40×3=120 秒

关思考可量化：同一次检索、8888 字上下文，开思考 82.27 秒、4410 个输出 token（思考 4103、正文 678 字），关掉后 8.77 秒、464 token、正文 948 字。端到端从 79.29 秒降到 32 秒。

3.5 AI 约束：提示词与防编造

3.5.1 上下文格式 `[文件名 第 N 页]` 的由来

检索层拼上下文的方式，决定了模型能否分清「哪句话是谁说的」。早期只带页码：

```
context = "\n\n".join("[第%s 页] %s" % (i["metadata"]["page_number"], i["text"]) for i in result)
```

10 个块可能来自 6 篇文献，只有页码时模型无从归属，只能揉成一段连续叙述。实测由此产生两个真实错误：

错误	表现
张冠李戴	把 CLIP 相关工作里提到的 Transformer，说成 Transformer 自己的做法
参数串篇	把《Scaling Laws》的 batch size（512 序列 × 1024 token）当成 Transformer 的超参数

改后每块带文件名：

```
context = "\n\n".join(
    "[%s 第%s 页] %s" % (i["metadata"]["source"], i["metadata"]["page_number"], i["text"])
    for i in result)
```

效果：同一份检索结果、同一个模型，模型自己把两篇的设置分开了，并注明「这属于另一篇论文的设置」。变的只是上下文前缀，检索结果一条没动。

代价有三条：

1. **上下文变长**。每块多一个文件名前缀，10 块就是十段重复的文件名。在 8888 字的输入里占比不大，但它是纯开销。
2. **文件名本身是噪声源**。文件名形如 `1706.03762v1.pdf`，编号极易被模型当成事实（「1706 是否代表 2017 年」）。所以加文件名必须同时加一条「不许从文件名、编号推断任何信息」，两条规则配套，缺一条就互相拆台。
3. **页码的权威性绑到了前缀上**。提示词要求「页码只能取自每段开头的 `[文件名 第 N 页]`」，一旦前缀格式改动（少个空格、换成全角括号），这条规则就静默失效。格式与提示词硬耦合，改一处必须改两处。

3.5.2 两条防编造护栏

跨文献问答最常见的失败不是「答错」，而是「答了另一篇」。护栏针对的就是这个：

场景	要求	防的错误
点名的文献在参考资料里完全没有	必须单独写一行「（本次检索未返回该文献）」	拿别的文献顶替，用户以为问到了
目标文献只被别的论文顺带提到（如相关工作里一句）	写明「仅《文件名》提到它，没有它自己的原文」	据此描述它的做法，实际是二手转述

两条都是硬约束，但只靠它们会走向另一个极端——整篇拒答。因此还有一条边界：

> *只要参考资料里有该文献的一部分内容，就先把有的写出来，缺的部分注明「资料中未给出这项」；只有一块相关原文都找不到时，才输出「参考资料未提及」。*

这条边界有直接证据：`max` 那次 41.81 秒、1610 个输出 token、正文只有 7 个字，全部内容是「参考资料未提及」。第一版护栏没有这条边界，把「超参数」那道题整题拒答了——库里其实有部分答案。护栏的目标是**不编造**，不是**不回答**；把「不全」当成「没有」，是把可用性问题伪装成准确性问题。

3.5.3 改前改后对照

维度	改前	改后
上下文前缀	`[第 N 页]`	`[文件名 第 N 页]`
跨文档归属	靠模型自己猜，实测串篇	前缀显式给出，模型自行分开
点名文献缺失	无规则，容易拿别的文献顶替	单独一行「（本次检索未返回该文献）」
仅被顺带提到	无规则	标注「仅《文件名》提到它，没有它自己的原文」
部分覆盖	倾向整篇拒答	先答已查到的，缺的注明「资料中未给出这项」
典型错例	《Scaling Laws》的 batch size 被当成 Transformer 的超参数	两篇设置分开陈述，并注明归属

3.5.4 其余约束各自防什么

规则	防的错误
结论必须有原文支撑	用先验知识补细节，答得对但库里没有
页码只能取自工具返回	自己推算页码，看起来精确其实是编的
查不到就回固定短语	用常识作答，用户无法分辨「有据」与「无据」
禁止从文件名/编号推断	`1706.03762` 是 arXiv 编号，不是 2017 年
参考文献年份不算本篇	被引文献的发表年极易被误当成目标论文的年份
一次只调一个工具	并行调用导致上下文拼接顺序不可控
联网内容必须标注出处	把网上结论说成本地文献的内容
同一轮最多 6 次 `local_search`	反复换关键词试同一篇，耗尽时间仍不覆盖

其中「最多 6 次」是**提示词软约束**，模型不总是严格遵守——无上限那次实验就出现了 19 次检索。它的作用是给模型一个预算感。

3.6 存储设计与一致性

3.6.1 六份存储

存储	内容	谁读
`chroma/`	727 个块的向量 + 元数据（`source`、`page_number`）	检索链

存储	内容	谁读
chroma_summary.json	每篇首页截断的摘要	相似判定时给模型看的材料
chroma_structured.json	标题、作者、表、图、公式、参考文献、附录 + 逐页原文	展示页、翻译、list_documents
bm25.pkl	727 条分词结果 + 每块来源	BM25 召回
md5.txt	文件指纹	查重第一层
embedding.json	183 页的页级向量	查重第二层

分成「块」与「整篇」两套是有意的：向量库存检索单位，chroma_structured.json 存展示与翻译单位。把结构化信息塞进块元数据，一块就会携带整篇的表和图，体积失控。

3.6.2 写入顺序，以及它不是原子的

```
chroma → chroma_summary.json → chroma_structured.json → bm25.pkl → md5.txt → embedding.json
```

顺序刻意把「这篇已入库」的**登记动作放在最后**（md5.txt、embedding.json）。若提前登记，中途失败会留下「账本里有、数据里没有」的幽灵记录——重传时 md5 拦不住（指纹已写），相似度那层却拿它和「自己」比，相似度必然 1.0，于是进入相似文档分支，去读一个从未写成的摘要文件，报「读取旧文档摘要失败」。真实原因是上一次入库没写完。

必须诚实标注：这六次写入不是原子的，也没有事务或回滚。 进程在任意两步之间挂掉（云端重启、容器被杀、磁盘写失败），都会留下前后不一致的状态。已知表现是：磁盘上有 7 个文件，但系统判定「没有相似文档」——md5.txt 认这篇，chroma/ 里却没有它的块。云端重启后出现过。

当前的处理是**事后修复**：读到缺失的摘要时抛出明确提示，让用户删掉 embedding.json、md5.txt 和 chroma/ 后重传。正确做法是写临时目录再原子替换，或引入写入清单做启动校验——列为后续工作。

3.6.3 删除必须六处联动

delete_file 依次清理 embedding、摘要、向量库、BM25、md5、结构化文件。漏一处就产生「删不掉的幽灵」：只删向量库没删 md5.txt，再传同一篇会被指纹拦下，但库里一片都没有，表现是「传了但搜不到」。

BM25 那一段按文件名过滤时，必须取字典字段再比较：

```
for token, src in zip(self.bm25_corpus, self.bm25_chunk_source):
    if src["source"] != filename:      # src 形如 {"source":…, "page_number":…}
```

早期写成 src != filename（拿字典与字符串比较），条件永不为真，一条都删不掉，删完文档后 BM25 仍能召回它的块。这两份列表是**并行**的，必须在同一循环里同时过滤，只删一边会让页码整体错位。

3.6.4 单进程约束

bm25.pkl 是整份覆盖写的（pickle.dump 全量），BM25Okapi 也不支持增量添加——每次入库都是整份重建。两个进程同时入库会互相覆盖，并使 BM25 索引条数与向量库块数不一致；检索层默认两者一一对应，一旦错位就会取到错误的 metadata，页码全错。

因此系统必须单进程运行。727 块规模下重建只要毫秒级，代价可接受；语料涨到几千块时需要换增量方案。

第 4 章 工程实现与质量保障

这一章不列功能清单，只回答几个工程问题：一次入库要写六处存储，靠什么保证它们不打架（4.1）；一次问答从 79.29 秒压到 32 秒，时间和 token 花在哪（4.2）；调试期间真实撞上并修掉的九个缺陷（4.3）；以及这套东西怎么部署、代码量有多少（4.4、4.5）。

所有数字都来自实测记录与代码常量。拿不到统计口径的地方直接写「未统计」，不做估算。

4.1 入库事务与幂等设计

4.1.1 六处存储与它们的写入顺序

一篇 PDF 进来，最终要落到六个地方。这六处没有事务保护，中途任何一步失败都会留下不一致状态，所以设计上能做的不是「消除窗口」，而是「把窗口压到最小、把顺序排对」。

顺序	存储位置	内容	失败后的可见症状
①	`chroma/`	向量块，主键 `{文件名}_p{页码}_{块号}`	检索不到这篇，但文档列表里有
②	`chroma_summary.json`	每篇前 2000 字摘要，相似度判断时给模型看	判相似时报「读取旧文档摘要失败」
③	`chroma_structured.json`	规则抽取的整篇结构（标题/作者/表/公式/参考文献）	详情页显示「没有结构化结果」
④	`bm25.pkl`	分词结果 + 每块的来源页码	只能向量召回，关键词精确匹配失效
⑤	`md5.txt`	「哪些文件入过库」的账本	重传时第一道闸拦不住
⑥	`embedding.json`	每篇的页级向量，查重与「已入库」的凭据	重传被误判成与「自己」相似

写入顺序是刻意的：`chroma → summary → structured → bm25 → md5 → embedding.json`。前四处是「数据」，后两处是「登记」。**登记必须最后写**，因为库里所有幂等判定都读登记：md5 命中就认为这篇完整入库过，embedding 里有记录就认为这篇有可比对的指纹。如果登记提前落盘，一旦后续步骤失败，系统会坚定地认为文档已存在，而摘要、向量、索引都不完整——这就是 4.3 第 3 条那个「7 个文件但不报相似」的来源。

4.1.2 第一道闸：md5 内容去重

上传入口拿到的不是字节流而是 base64 字符串，同一份内容 base64 编码结果稳定，所以 md5 可以当内容指纹用。这一道闸放在解析之前：

```
str_hashlib = get_str_md5(base64_str)
if check_md5(str_hashlib):
    return False, filename, filename     # 完全重复，连 PDF 都不打开
```

命中时不做任何计算：不解析、不切块、不嵌入、不写库。省下的是整条解析与向量化链路。

`upload_str_pdf` 的返回值有三种形态，页面按类型分支，这是整个上传流程最容易被误读的地方，因此单独列一张表：

返回值	含义	是否已写入	页面行为
`(False, 原名, 原名)`	md5 命中，内容完全重复	未写入	提示「MD5 已存在，跳过本次上传」
`(True, None, 新名)`	无相似文档，入库完成	已写入	提示「文档入库完成，无相似文档」
`(dict, 旧名, 新名)`	命中相似文档，等用户决定	**未写入**	挂起 `pending_task`，展示智能体建议

第三种形态必须强调「未写入」：此时解析结果只暂存在内存字典 `temp_pdf` 里，用户点按钮之前库里什么都没有。这也是「取消」能够做到零副作用的前提。

4.1.3 命中相似文档后的三种处理

相似度判定通过后，把新旧两篇的摘要和相似度数值一起交给智能体，要求它只输出三个字段的 JSON（`sim_desc` / `action` / `reason`），再解析成按钮上方的建议文案。解析失败不算致命，页面直接报「智能体返回结果解析失败，请重试上传」，不入库。

三个按钮对应三条数据路径：

用户选择	写入动作	删除动作	结果
覆盖旧文档入库	`real_save_pdf` 写新文档	同名：先删后写；不同名：写成功后再删旧	库里仍是一篇
两篇全部保留	`real_save_pdf` 写新文档	不删	两篇并存，各自独立 md5
取消本次上传	不写	不删	只丢弃 `pending_task`，无需回滚

「取消」不做 `st.rerun()` 是有意的：rerun 会把刚写出的「已取消」提示立刻冲掉，用户看到的是「点了没反应」，然后往往再传一遍，最后库里多一份。

4.1.4 先存后删：一次真实的双丢

最初的覆盖分支是无条件「先删旧的、再写新的」：

```
delete_file(old_filename)
real_save_pdf(new_filename, base64)
```

删除成功了、写入失败了（解析异常、嵌入超时、云端中断），旧文档已经没了，新文档也没落盘，用户的观感是「选了覆盖，两边都没了」。修正后按文件名是否相同分叉：

```
same_name = (task["old_filename"] == task["new_filename"])
if same_name:
    delete_file(old_filename)          # 同名：块数可能变少，必须先删干净
saved, _, _ = real_save_pdf(new_filename, task["pdf_base64"])
if saved is not None and not same_name:
    delete_file(old_filename)          # 不同名：写成功才删旧
```

两类顺序相反，原因不同：

- **同名**：块 id 是 `{文件名}_p{页码}_{块号}`，是确定性的，`add_texts` 按 id 覆盖。重传后如果切出的块数变少，多出来的高编号旧块不会被覆盖，而是残留成「搜得到但内容已过期」的脏数据。所以必须先删干净。
- **不同名**：新旧文件的 id 前缀不同，本来就不冲突，没有必须先删的理由；反过来要保住旧文档，直到新文档真的写进去。

顺序修正只能缩小双丢窗口，不能消除。真正的原子化方案（先写临时目录再整体替换、或引入单文件数据库）没有实施，这一条与 4.3 第 3 条同属已知架构缺陷。

4.1.5 `pending_task`：防重复提交

Streamlit 的执行模型是「每点一次按钮，整个脚本从头重跑一遍」。上传组件里的文件在重跑后仍然在，如果不拦，一次上传会被解析多次，每次都重新解析、重新嵌入。

拦法刻意做得很窄：只有「已经挂起、正在等用户确认，且挂起的文件名就是当前文件名」这一种情况跳过，其余一律照常走。

```
pending = st.session_state.pending_task
if pending is not None and pending.get("new_filename") == name:
    st.caption("这个文件正在等待确认，请看下方面板")
else:
    ...  # 正常解析
```

为什么不用「文件名 + 大小」当指纹：那会把合法重传一起拦掉。已入库的文件本来就由 md5 兜住——只比一次 md5，不解析、不嵌入，成本可以忽略；而用户确实想重传一次时，系统必须让他传。

`pending_task` 里除了两个文件名和建议文案，还存了一份 `pdf_base64`。理由是时间：从挂起到用户点按钮之间可能过了几十秒，中间页面重跑过、会话可能重建过，`temp_pdf` 里那份解析结果未必还在。按钮分支把 base64 一起传进 `real_save_pdf`，缓存失效时就重新解析一遍——慢，但不会出现「点了按钮，提示解析结果已丢失」这种死路。缓存从「前提」降级成「能省就省的快路径」，是这一处设计的核心。

4.2 AI 优化策略：延迟与成本

4.2.1 三方对比：时间花在思考上

同一份检索结果、同一段输入（8888 字上下文）、同一个问题，只换模型和思考开关：

配置	耗时	输出 token	其中思考 token	正文
flash 默认（开思考）	82.27 s	4410	4103	678 字
flash 关思考	**8.77 s**	**464**	**0**	**948 字**
max 默认（开思考）	41.81 s	1610	1604	7 字（只回了「参考资料未提及」）

这张表里有三个可以直接算出来的比例：

- flash 开思考时，思考 token 占输出 token 的 **93%**（4103 / 4410）；max 开思考时是 **99.6%**（1604 / 1610）。所谓「八成以上 token 花在思考上，正文只占一成」，实际比八成还要极端。
- 关掉思考后输出 token 只剩原来的约 **10.5%**（464 / 4410），而正文字数从 678 涨到 948，反而长了四成。
- max 那一行最能说明问题：1604 个思考 token 换来的正文是 7 个字，而且是一句拒答。思考没有让它答得更好，只让它犹豫得更久。

对这个任务来说，思考是纯支出。这一层要的是「照着检索到的原文复述并标页码」，不需要模型自己想；想得越多，越容易越过原文去补常识，而补常识恰恰是提示词明令禁止的。

4.2.2 端到端收益

场景	改前	改后
一次问答（问「attention is all you need 中，超参数是多少」，本地）	79.29 秒	**32 秒**
同一问题，云端实测	—	**34 秒**
一次按章节翻译	—	**19 秒**

检索本身不是瓶颈：检索延迟均值 0.50 秒，最大 0.96 秒。32 秒里绝大部分仍然是模型生成，所以 4.2.1 的那一刀才能带来数量级的差异——把生成从 82 秒压到 8.77 秒，端到端才有 79.29 → 32 的可能。

4.2.3 `timeout=45, max_retries=0` 的由来

换模型之前还有另一个问题：一次问答会莫名其妙挂三分钟以上，界面停在「正在检索文献中…」，既不返回也不报错。

根因是两处叠加。模型调用没有设 `timeout`，请求挂住就无限期等；同时 `max_retries=2`，SDK 会把一次挂住原地重试两次。于是一次卡住被放大成 **40×3 = 120 秒**，而这个智能体一次问答要调两三次模型，叠起来就是三分钟以上。

```
model = ChatOpenAI(
    model="qwen3.8-flash",
    timeout=45,
    max_retries=0,
    extra_body={"enable_thinking": False},
)
```

重试次数取 0，不是因为它更「稳」，而是因为这条链路的失败模式不是偶发瞬时错误，而是请求挂住。重试对前者有效，对后者只是把等待时间乘以次数。快速失败之后页面立刻给出可操作提示，用户再问一次的成本远低于干等两分钟。

同样的三个参数在 `agent.py` 与 `rag.py` 里各写了一遍，这是有意的重复：`rag.py` 那一次是整条链路里最慢的生成（输入七八千字），只掐外层不掐内层，等于没掐。

4.3 缺陷排查记录

以下九条都是实际调试中撞上的，按「现象 → 定位 → 修法」记录。先说共性：其中三条的根因是同一个前提——**Streamlit 每次交互都会把脚本从头重跑，内存、屏幕、会话历史都不保证还在**。凡是把状态存在模块级变量或「上一次渲染的结果」里的写法，都会在这里翻车。

#	现象关键词	影响面
1	`KeyError` at `del self.temp_pdf[filename]`	覆盖/保留按钮点击即崩
2	选覆盖后新旧文档双丢	数据丢失
3	7 个文件但不报相似、检索不到	数据一致性
4	`Expected ASGI message 'http.response.start'`	整站不可用
5	一次问答挂三分钟以上	可用性
6	模型编造「我查过了但没拿到返回」	回答可信度
7	问答页看起来是白板	交互可用性
8	联网开关关着却联网作答	功能正确性
9	一轮问答检索了 22 次	延迟与成本

4.3.1 `KeyError` at `del self.temp_pdf[filename]`

现象：用户点「覆盖旧文档入库」，页面直接抛 `KeyError`，按钮还留在原地。用户看到的是「点了没反应」，于是再点一次，或者重新上传一遍。

定位：`real_save_pdf` 末尾是 `del self.temp_pdf[filename]`，默认「上传时缓存的那份解析结果一定还在」。`temp_pdf` 是纯内存字典，云端休眠唤醒、会话重建、页面重跑都可能让它空掉。键不存在时 `del` 直接抛异常，异常发生在写库之前，所以库里什么都没变——但用户不知道。

修法：改成 `pop(filename, None)`，取不到就走兜底路径，用传进来的 base64 重新解析、重新嵌入；按钮分支必须把 `pdf_base64` 一起传进来。缓存从「前提」降级为「快路径」，这是 4.1.5 里存 base64 的直接原因。

4.3.2 「选了覆盖，结果两边都没了」

现象：覆盖分支提示操作完成，回到文档列表，新文档不在、旧文档也没了；有时只剩一个搜不到内容的空壳。

定位：原逻辑无条件「先 `delete_file(old)`、再 `real_save_pdf(new)`」。删除是不可逆的，写入是可能失败的（解析异常、嵌入超时、云端中断）。一旦写入失败，旧文档已经被删掉，构成典型的双丢。

修法：顺序按文件名是否相同分叉，同名先删（块 id 会残留，见 4.1.4），不同名先写成功再删旧。写入返回 `None` 时页面明确报「待入库的解析结果已丢失，请重新上传这个文件」，而不是照样提示成功。

4.3.3 「7 个文件但不报相似」

现象：文档列表显示 7 个文件，其中一篇是刚传的、与库里某篇高度相似，上传时却没有触发相似度判断；检索时这篇也查不到内容。

定位：六处存储非原子写入。云端重启如果发生在写入中途，六处会停在不同步骤上，出现「`chroma_structured.json` 里有这篇、`embedding.json` 里没有」这类组合。相似度判断只读 `embedding.json`，页面列表读的却是 `chroma_structured.json`，两边各信一份，表现就是「列表里有、查重看不见、检索查不到」三者同时成立。

修法：一是把「这篇已入库」的登记（`md5.txt` 与 `embedding.json`）压到流程最后写，中途失败时库里不会留下半条登记；二是删除时六处一起删，`delete_file` 逐项清理并在注释里写明漏一处的症状（例如只删 Chroma 不删 md5，用户重传会被 md5 拦下，但库里其实一片都没有）。原子性本身没有解决，这是已知架构缺陷。

4.3.4 ASGI 协议的启动错误

现象：本地一切正常，云端每次重建后整站打不开，日志里是

```
RuntimeError: Expected ASGI message 'http.response.start', but got 'http.response.body'
```

定位：`starlette` 没有在依赖里钉版本。Streamlit 只声明了下界不写上界，云端每次重建都会装到当时最新的 `starlette`，与 `streamlit` 对不上时 ASGI 协议层就报这个错。这类问题与业务代码完全无关，重装一次可能复现、也可能消失，非常容易误判成「云端环境不稳定」。

修法：`requirements.txt` 里钉 `starlette==1.3.1`，并在该行上方写明原因，避免后来者顺手升级又踩回去。

4.3.5 一次问答挂三分钟

现象：界面停在「正在检索文献中…」，最长超过三分钟，不返回也不报错。

定位：模型调用无 `timeout`，请求挂住就无限等；同时 `max_retries=2`，一次卡住被放大成 40×3=120 秒。一次问答要调两三次模型，叠加即三分钟以上。

修法：`timeout=45`，`max_retries=0`，`agent.py` 与 `rag.py` 两处一起改。推导过程见 4.2.3。

4.3.6 模型编造「我查过了但没拿到返回」

现象：某一轮抛异常后，会话历史里只留下一个用户提问、没有对应回答。用户下一轮追问，模型给出「我查过了但没拿到返回」这类理由——它并没有查过，它只是看到了历史里那个空档，然后为缺口编了一个解释。

定位：异常直接冒到 Streamlit 层变成一片红字，同时没有写进 `st.session_state["history"]`。对话历史出现「有 user、无 assistant」的洞，模型唯一能做的就是解释这个洞。

修法：流式循环外面包 `try/except`，异常写进历史并以 `error` 展示，文案明确区分「没答上来」和「没跑完」：

```
except Exception as e:
    failed = e
    answer = "这一轮没有跑完（%s）。这不是查不到，是没跑完，请再问一次。" % type(e).__name__
    placeholder.error(answer)
```

异常路径刻意不走非流式的 `invoke` 兜底——那会把整条链再付一遍，耗时翻倍，而且大概率还是失败。

4.3.7 问答页看起来是白板

现象：多轮检索的问题上，回答区先出现几行灰色过渡语，主区域只剩一个孤零零的 ▌ 光标，看起来像页面空白。用户以为回答丢了，重新提问。

定位：两处小问题叠加。模型决定调工具时，代码把已经吐出的过渡语搬进灰字区，然后 `answer = ""` 清空变量。但清空变量不等于清屏，屏幕上原来的字还在；接着下一段流式碎片如果恰好是换行，`answer.strip()` 为空却仍然调用了 `placeholder.markdown(...)`，于是渲染出一个只有光标的空块。

修法：两条一起改。空白内容不渲染，只有 `answer.strip()` 为真才画；清空回答区的同时立刻补一句提示，让屏幕始终有明确状态：

```
if answer.strip():
    log.append(answer.strip())
    answer = ""
    placeholder.info("正在继续检索…")   # 清空的同时补状态，不留白板
```

灰字区同时只保留最近几条，避免过渡语越堆越长。

4.3.8 联网开关关了，回答却来自互联网

现象：问答页的「联网搜索」开关处于关闭状态，开关下方明明白白写着「本次回答：仅本地文献库」；问一个库里绝不可能有的问题（例如某商业模型的训练卡数），却拿到一整段带媒体报道名、发布日期和链接的答案，页脚显示「共检索 22 次，用时 250 秒」。本地文献是六篇机器学习论文，其中不可能有这段内容。

定位：开关只做了一件事——中间件 `pick_prompt` 按 `use_web` 拼提示词。但 `web_search` 一直挂在 `create_agent(tools=[...])` 里，模型的工具列表里始终有它，schema 没有任何变化。**提示词里不写它，不等于它不可调用**：模型看到库里没有答案，就去调那个能查外部资料的函数。

第二个前提让「关着」也不安全。`tool_2.search_tools()` 查的是 `TAVILY_API_KEY`，而本机 `.env` 里写的是小写 `tavily_api_key`。Windows 的环境变量名不区分大小写，本地照样查得到，联网实际是通的——正是 4.4.3 要讲的 Linux 大小写坑的镜像。于是提示词说「不许联网」，工具却能联网，两者谁也没拦住。

同一轮 22 次调用还暴露了另一件事：提示词里「最多调用 6 次」没有任何代码强制，模型可以一直试——那是 4.3.9 要单独说的问题。

修法：把开关下沉到工具内部。`web_search` 增加一个隐藏参数 `runtime: ToolRuntime`（由运行时注入，不进模型的工具 schema），进函数先读本次调用的 `context`：

```
if not (runtime.context or {}).get("use_web", False):
    return "联网搜索未开启（用户没有打开联网开关）。不要用你自己的知识回答，直接回答「库中没有相关资料」。"
```

这样开关有了两层：提示词决定模型该不该用，工具决定它能不能用。关掉时多一次往返，换来的是「界面写什么，系统就做什么」——这条缺陷的教训不是漏了校验，而是**开关必须加在能力上，加在提示词上只是建议**。

还有一个值得记的观察：这个缺陷在线上永远复现不出来。云端缺少大写的 `TAVILY_API_KEY`，`search_tools()` 返回 `None`，联网本来调不动，行为一直是对的。它只在本地发作，而本地正是录演示视频的地方。

4.3.9 一轮问答检索了 22 次

现象：同一个问题，页脚显示「共检索 22 次，用时 250 秒」。4.3.8 那次联网问答就是这样，中间没有任何东西拦住它。

定位：上限只写在提示词里——「同一轮最多调用 local_search 6 次」。提示词是**建议**，代码里没有任何地方计数，模型想调几次就调几次。本地查不到时，它会一轮一轮换关键词重试，每次都真的走一遍向量检索加 BM25，22 次就是两百多秒。

修法：分两层，各管一件事。

第一层管「查不了」，落在工具内部：`local_search` 和 `web_search` 进函数先数一遍本轮已完成的检索次数，到 6 次就直接返回一句话，不碰向量库、不发网络请求。计数不存模块级变量，而是从消息历史往前数到最近一条用户提问——历史本来就在，不依赖任何额外状态，也不会因为 Streamlit 重跑脚本而串会话或忘记归零。

这段逻辑放在独立的 `search_budget.py` 里，两个工具都从它导入。放在任何一个工具文件里都会让另一个反向依赖；而「6」这个数只该有一处定义——`agent.py` 的中间件、两个工具、页面上的提示文案，四处读的都是同一个常量。

第二层管「停下来」。这一步比想象的麻烦，试错了两次：

- 只返回拒绝文本不够。把上限临时压到 2 实测：2 次正常检索之后，模型把拒绝当成临时故障，又连调了 13 次被拒的调用，中间还插了 4 次翻译，一轮白烧 57 秒。它需要的是「没有工具可用」，不是「工具说不行」。
- 在模型请求里摘掉工具、同时加 `tool_choice="none"` 也不够。日志确认工具确实从请求里摘掉了（只剩翻译和列表两个），但这家 provider 照样往一个已不存在的工具上发调用，而工具节点手里还握着完整工具表，于是照跑。模型层的约束到不了执行层。

最终用的是框架自带的 `ToolCallLimitMiddleware`：

```
limit_search = ToolCallLimitMiddleware(
    tool_name="local_search",
    run_limit=MAX_RETRIEVAL,    # 按「一轮」算，不是按会话
    exit_behavior="end",        # 超限直接结束这一轮，而不是把拒绝丢回去
)
```

验证：上限压到 2 时，工具调用序列是 `list_documents → local_search ×3`，第 3 次触发后这一轮立即结束，没有后续重试；恢复成 6 跑「这几篇论文分别用了什么方法」，正好 6 次检索、正常答完、没有触发上限提示。

残留问题：框架结束这一轮时写下的最终消息是一句英文内部提示（`'local_search' tool call limit reached: run limit exceeded`），会原样刷到页面上。页面层把它换成了中文说明。真正干净的做法是超限时用已检索到的内容直接作答，那需要自己在工具里拼答案，代价是绕过 `rag.py` 的生成链，暂未做。

4.4 部署与配置

4.4.1 部署形态

项目	内容
平台	Streamlit Community Cloud（免费实例）
容器内路径	`/mount/src/research-paper-assistant/`
入口文件	`web.py`（上传与结构化查看）；问答页在 `pages/app.py`，Streamlit 自动生成侧边栏入口
依赖	`requirements.txt` 全量钉版本
密钥注入	平台 Secrets → `os.environ` → 各模块 `os.getenv`

项目	内容
向量库持久化	persist_directory="./chroma"，相对容器工作目录
在线演示	https://research-paper-assistant-yx.streamlit.app/

免费实例会休眠：长时间无人访问后，首次打开需要等待唤醒。README 的在线演示一节写明了这一点，避免评审把唤醒等待误判成服务不可用。

仓库内随带样例入库产物。否则重建后的容器是一个空知识库，首次访问必须先自己上传论文才能问第一个问题，演示路径上凭空多出一道门槛。产物总体积未统计。

4.4.2 Secrets 注入的位置比内容更重要

Streamlit Cloud 的 Secrets 不会自动变成环境变量，而项目里所有模块读的都是 os.getenv。所以入口文件的第一件事是把 Secrets 逐项灌进 os.environ：

```
import streamlit as st
import os
try:
    for k in st.secrets:
        os.environ[k] = str(st.secrets[k])
except Exception:
    pass          # 本地没有 secrets.toml，读的是 .env

from knolege import *
```

关键不在这一段代码，在它的位置：**必须在 `from knolege import *` 之前**。vctor 在 import 的那一刻就会构造 Knolege() 实例，构造函数里立刻要用 DASHSCOPE_API_KEY 建 embedding 客户端、用 qwen_url 配模型。注入放在 import 之后，构造函数拿到的就是空 key——现象是启动即鉴权失败，而不是「没配 key」这种一眼能看懂的提示，排查成本高得多。

try/except 也是必须的：本地没有 secrets.toml，遍历 st.secrets 会抛异常，此时应该安静地退回 .env（load_dotenv() 在各模块里都调了）。云端和本地两条配置来源，靠这个 except 分流。

4.4.3 Linux 上变量名必须大写

.env.example 里专门写了一条：变量名区分大小写，Linux/macOS 上写错就读不到。

- TAVILY_API_KEY 必须全大写，langchain-tavily 只认这个名字。
- Windows 上环境变量不区分大小写，写成 tavily_api_key 照样能跑；换到 Linux 就报缺 key。
- 这个错误的报错点不在启动时，而在用户第一次打开联网开关、真正调用联网工具时，很容易被当成「联网功能没实现」。

同类问题还有 DASHSCOPE_API_KEY：一个 key 同时服务三处——qwen3.8-flash 对话、text-embedding-v4 向量化、结构化抽取的兜底模型。任何一处拼错，症状都不一样，但都不会直接指出是变量名的问题。

4.4.4 依赖钉版本

requirements.txt 里所有包都写 ==，没有 >=。这不是洁癖：4.3 第 4 条就是浮动版本造成的整站不可用。几个关键项：

包	版本	作用
streamlit	1.59.2	前端与页面执行模型
starlette	1.3.1	Streamlit 的 ASGI 服务层，必须钉死
langchain	1.3.10	Agent 与工具编排
langgraph	1.2.6	图执行与 checkpoint
chromadb	1.5.9	向量库
pymupdf / pymupdf4llm	1.28.2	PDF 解析与 markdown 化

fastapi、uvicorn 目前装上但未使用，为服务化迁移预留。

4.5 代码规模

统计时间 2026-11-05，口径为项目根目录下全部 .py 文件：

指标	数值	占比
.py 文件数	21	—
总行数	4577	100%
代码行	3429	74.9%
注释行	667	14.6%
空行	481	10.5%

占比按总行数计算（如 3429 / 4577 = 74.9%）。**按文件拆分的行数未统计**，本节只给总量，不按文件拆分——分文件统计需要单独的脚本口径，目前没有做。

21 个文件清单（可用于核对统计口径）：`agent.py`、`bm25_score.py`、`config.py`、`fs.py`、`glossary.py`、`graph tree.py`、`knolege.py`、`md5_saved.py`、`model_extractor.py`、`pages/app.py`、`paper_extractor.py`、`promot.py`、`rag.py`、`search_budget.py`、`tool_1.py`、`tool_2.py`、`tool_3.py`、`tool_4.py`、`translation.py`、`vctor.py`、`web.py`。

注释占 14.4%，明显高于「解释这行在做什么」所需的密度。相当一部分注释记录的不是语法，而是顺序约束和失败模式——`_save_to_all_db` 结尾解释「为什么登记必须最后写」，`web.py` 上传分支解释「为什么按钮段必须与 `if file is not None` 平级」「为什么不能用文件名+大小当指纹」，`delete_file` 里解释 BM25 的两份列表必须同步过滤。这些注释就是 4.1 与 4.3 里那些坑的现场记录，删掉它们，下一个维护者会在同样的位置再踩一遍。

规模本身没有解释力，但它能回答一个必然会被问到的问题：这套系统是不是只是把框架拼起来。21 个文件里，六处存储的写入顺序、相似文档的三分支处理、`pending_task` 的跨重跑状态、入库与删除时的六处同步，都是围绕具体故障长出来的，框架不提供现成答案。

第 5 章 测试与部署

本章的实验对象是仓库自带的六篇论文：共 183 页、727 个文本块。评测分三层：检索层看命中与排序，编排层看工具调用能否覆盖全部文献，端到端层看一次问答的耗时与 token 开销。所有数字均来自实际运行记录；记录中没有保留的项目在文中标为「未统计」，不做推算。部署形态与 Secrets 注入见 4.4，本章只给上线后的验证结果与实验数字。

5.1 测试方法与评测口径

检索评测的第一个问题是「标准答案从哪来」。做法是先把问题限定在可核对的事实上：从六篇论文的正文中选出 37 个事实型问题，用中文提问、英文语料作答，每题标注答案所在的（文件名，页码）作为判定依据。这样「答对」不依赖人工对答案质量的印象，而依赖「检索结果是否落在标注页」这一可复查的事实。37 题的题型构成与标注复核通过数未统计，这里只交代判定口径。

指标定义如下：

表 5-1 检索指标定义

指标	定义	回答的问题
hit@1	返回结果第一条即落在标注页的题目比例	排序头部是否准
hit@10	前十条中任意一条落在标注页的题目比例	送进上下文的材料够不够
MRR	首个命中位置的倒数，对 37 题取平均	命中之外，排得够不够靠前

三个指标里，hit@1 与 MRR 衡量排序质量，hit@10 衡量召回上限。之所以把 hit@10 作为主口径而不是端到端准确率，原因有两点。其一，系统的检索环节固定取 Top-10 拼上下文，用户实际拿到的就是这十条，hit@10 直接对应「答案在不在模型眼前」。其二，端到端准确率把检索、上下文拼接、生成三步混在一起，一旦答错无法判断是没检索到还是生成时没用上；在 37 题的规模上，这种混合指标读不出改进来自哪一层。因此本章先看 hit@10 与 MRR，把生成质量留到 5.5 节的整题实验单独观察。

方法上的局限需要先说明：37 题以事实查找为主，不覆盖「这篇论文的核心贡献是什么」这类开放题；每题只判「有没有检索到正确页」，不评价表述；实验为单次运行，37 题的样本量下，0.02 量级的 MRR 差异应视为噪声，后文不做过度解读。

5.2 结构化抽取结果

六篇论文的页数分布不均，最长 48 页，最短 15 页：

表 5-2 六篇论文的页数

论文序号	1	2	3	4	5	6	合计
页数	15	30	48	26	34	30	183

单篇的块数与抽取条目数未逐篇统计，只保留了合计口径。页数差异会直接影响切块数量与可召回的内容量，但逐篇的块数分布没有记录，后文不按篇比较块数。

表 5-3 结构化抽取合计

项目	数量
论文	6 篇
页数	183 页
文本块	727 块
其中：表格块	82 块
其中：正文块	645 块
表格	102 张
图	138 张
参考文献	546 条
附录	84 个
作者	78 位

两个口径需要区分：102 张表分布在 82 个表格块中，表数与块数不是一回事——一张宽表可能被切分到两个块，一个块也可能包含多张小表；同理，84 个附录是识别到的附录条目数，不等于「六篇论文共有 84 篇附录」。表格块只占 727 块中的 82 块，正文块 645 块是绝对主体，这决定了检索的主要对手是长正文，而不是结构化表格。

5.3 检索对比实验

三路对比在同一个 37 题集、同一份 727 块索引上完成，融合权重固定为 α=0.20、返回 Top-10：

表 5-4 三路检索对比（37 题）

方法	hit@1	hit@10	MRR

方法	hit@1	hit@10	MRR
纯向量	15/37	33/37	0.532
纯 BM25	10/37	26/37	0.423
融合 α=0.20	**17/37**	**33/37**	**0.585**
oracle 上界	—	—	0.642

融合在 hit@1 上比纯向量多 2 题，MRR 高 0.053，说明两路互补的增益是真实的；但 hit@10 与纯向量同为 33/37，从总量上看融合没有多救回任何一题——它改善的是排序，不是召回边界。纯 BM25 单独使用最弱（hit@1 10/37、MRR 0.423），符合「中文提问、英文原文」的设定：字面匹配只能命中问题里夹带的英文标识符，覆盖不了语义改写。oracle 上界 0.642 表示每题都取两路中较好的那一路时的 MRR，融合距离这个上界仍有 0.057，这部分差距不是调权重能补上的（见 5.4）。

33/37 还意味着有 4 题在 Top-10 内没有命中。这 4 题究竟是切块时答案被切断、页码标注偏移，还是问题本身需要跨页综合，现有记录没有逐题归因，标为未统计。能确定的只是：hit@10 在这一版系统上已经摸到天花板，仅凭 α 与 K 的调整，现有数据看不到能改变这 4 题的迹象，改进要回到解析与切块层。

候选池大小是另一个常被怀疑的因素。把候选集 K 从 10 扩到 100：

表 5-5 候选集 K 对 MRR 的影响

K	10	25	50	100
MRR	0.550	0.550	0.550	0.550

四档完全不变，说明瓶颈不在候选池大小：扩池只是把更多低分块放进排序，头部十名的构成没有变化。需要说明的是，这组实验的绝对 MRR 是 0.550，与表 5-4 中 α=0.20 的 0.585 不在同一配置下，该组实验的参数配置未记录，因此这里只读「K 不敏感」这一趋势，不把 0.550 与 0.585 直接相减。

5.4 融合机制分析

融合公式为 `final = α · BM25_norm + (1 − α) · cos`，α 控制字面匹配的权重。对 α 做扫描，得到四个可比较的点：

表 5-6 α 扫描（37 题）

α	MRR
0.10	0.603
0.20	0.585
0.35	0.547
≥0.60	0.485

曲线单调下降：BM25 权重越高，MRR 越低。原因在语料与提问的语言错位上——提问是中文，原文是英文，BM25 能匹配的主要是问题中保留的英文术语与标识符，权重一高，排序就被少数关键词命中主导。α 从 0.35 抬到 0.60 以上时 MRR 掉到 0.485，比 0.20 低 0.100，说明「多用关键词」的方向性判断是错的，不是参数没调细。

真正需要解释的是：最高点出现在 α=0.10（0.603），正式配置却取了 0.20（0.585）。理由有三条。第一，0.018 的差距在 37 题、单次运行的条件下落在噪声量级，不足以支撑「0.10 更优」的结论。第二，α=0.10 把 BM25 的权重压到很低，等于把第二路退化成微调；而 5.3 的三路对比显示纯向量在 hit@1 上只有 15/37，保留一部分字面匹配对含 `r=4`、`CIFAR-10` 这类短标识符的问题是必要的。第三，0.20 是在正式三方对比中完整测过 hit@1、hit@10、MRR 的配置（17/37、33/37、0.585），取它在证据上更完整，也更符合「融合要有实际权重」的设计意图。因此 0.20 是折中值，不宣称它是唯一最优。

α 扫描只记录了上述四个点，0.15、0.25、0.30、0.40 等中间取值未统计，曲线在 0.10 到 0.35 之间的具体形状无法判断，这也是不做更细调参的原因之一。

从机制上看，融合离 oracle 的 0.057 差距来自「一套固定权重应付所有题」这件事本身。有的题靠语义改写才召得回，希望 α 接近 0；有的题靠 `r=4` 这类短串，希望 α 高一些。线性加权只能给出一个全局折中，无法逐题切换。要接近上界，需要的是按题自适应地选路，或者改用只比较名次、不比较分数的排序融合（如 RRF），让两路的分数尺度不再直接相加。这部分属于后续工作，本章没有对应的实验数据，不展开评估。

5.5 工具与编排实验

这一节是本章的重点。检索指标只说明「给定一次查询能否找对」，不说明智能体是否会主动把六篇论文都查到。为此设计了一组整题实验，题目是「这几篇论文分别用了什么方法？各自的创新点是什么？」，要求覆盖全部六篇，对三种工具配置各跑一次：

表 5-7 六篇全扫题的三组配置对比

配置	工具调用	总耗时	覆盖	结果
无上限	19 次检索	279.46 s	6 篇	内容全，但过渡语混进答案，且谎称联网查到

配置	工具调用	总耗时	覆盖	结果
上限 6，无清单工具	10 次检索	83.93 s	2 篇	3 篇误报「库中没有」，实际都在库里
上限 6 + list_documents	1 次列目录 + 6 次检索	125.54 s	**6 篇**	六篇分节，标题全对

三行的差别不是「快慢」，而是三种失败模式。无上限那组把时间花到 279.46 秒、覆盖是满的，但答案里混入「让我先查一下」这类过渡语，还出现「已联网查到」的虚假陈述——19 次调用全是本地检索，模型把自己的检索行为描述成了联网。没有上限时，问题从「查不到」变成「查得太多且叙述失真」。

第二组加上了「最多 6 次检索」的提示词约束，耗时降到 83.93 秒，但覆盖从 6 篇掉到 2 篇，另 3 篇被报成「库中没有」——实际都在库里，模型只是凭前几次检索的结果就下了结论。它实际调用了 10 次，超过设定的 6 次：当时上限只是提示词软约束（现已改为代码强制，见 4.3.9）。所以「加上限」只解决成本，不解决覆盖，甚至会把「没查到」放大成「不存在」。

第三组只加了一个 `list_documents` 工具：它读 `chroma_structured.json`，返回「文件名 + 标题 + 页数」，不调模型、不检索，0.05 秒返回。模型先花 1 次调用拿到六篇的清单，再按清单逐篇检索 6 次，总耗时 125.54 秒，六篇全部覆盖，答案按篇分节。比无上限的 279.46 秒少一半以上，比「只有上限」的 83.93 秒多 41.61 秒——这 41.61 秒买到的正是从 2 篇到 6 篇的覆盖。

结论是：单纯加上限解决不了覆盖，解决覆盖的是「先知道有什么，再决定查什么」。清单工具的成本几乎可以忽略（0.05 秒），却把检索从一个开放动作变成在已知集合上的枚举。

多篇对比题也能看到同一机制：

表 5-8 三篇对比题的单次运行

题目	检索次数	耗时	答案长度	结构
Transformer、CLIP、LoRA 三篇的核心方法有什么区别	3 次	51.74 s	1142 字	三节 + 一句话关系小结

三篇对应三次检索，没有多余调用，51.74 秒与 1142 字都可接受。这题能一次成型，前提仍是模型知道这三篇在库里；缺少清单时，这类题会退化成表 5-7 第二组那种「只答出召回的部分」。

这组实验也解释了界面层的两条改动：把多轮检索中的过渡语从答案里剥到上方灰字区，否则会堆进答案；末尾显示「共检索 N 次，用时 X 秒」，让「谎称联网」这类叙述有可对照的凭据。两者不改变检索结果，但让工具调用过程可观察。

三组各只跑一次，耗时差值会受网络与排队影响；但覆盖 2→6 篇、误报 3→0 是结构性差异，不是单次波动能解释的。

5.6 端到端性能与成本

收益最大的一项优化是换模型并关闭思考模式，其耗时与 token 对比已在 6.4 分析，这里只看端到端：

端到端再看一次完整问答与一次翻译：

表 5-9 端到端耗时

场景	耗时
本地问答（优化前）	79.29 s
本地问答（优化后）	32 s
云端问答（同一问题）	34 s
按章节翻译一次	19 s

本地 79.29 秒降到 32 秒，降幅小于单次生成的 82.27→8.77，差额来自检索、上下文拼接、多次模型调用与网络等待，这部分未做逐段计时。云端 34 秒与之接近，说明免费实例的主要开销仍是模型调用而非部署环境。翻译 19 秒低于问答，因为它按章节取原文、一次成文，不必先检索再组织；此解释为定性，未分段计时验证。

检索本身的延迟不是瓶颈：

表 5-10 检索延迟

指标	数值
平均	0.50 s
最大	0.96 s
最小	未统计

平均 0.50 秒、最大 0.96 秒，相对 32 秒可忽略；所以优化重点在生成侧——检索再快一倍，端到端也只省不到一秒。

成本只有 token 口径：单次生成 4410→464，约原来的十分之一；项目未记录模型单价，不做金额换算。这组数字来自单次测量、上下文固定 8888 字，不应当作所有请求的均值；云端冷启动与网络抖动未统计，34 秒只代表一次实测。

第 6 章 技术创新点与应用创新点

本章分两部分。6.1 至 6.4 是**技术创新点**，四层四件事：检索层的分数可比性、生成层的上下文归属、编排层的检索前先验、成本层的思考开关。6.5 是**应用创新点**，四条都落在用户能直接感知的地方，与技术层不是同一类东西。6.6 说明与三类同类方案的差别与各自边界。

先把边界说清楚——没有算法层面的创新，不训练模型，不改 BM25 的打分公式。四处改动的共同结构是：先有一个具体的失败，再有一处可测的改动，最后有一个能复查的数字。凡是拿不出数字的地方，本章标注「未统计」，不用形容词补。

6.1 检索层：两路召回的可比性处理

两路召回的分数量纲不一致，是混合检索绕不开的第一个问题。向量那一路，Chroma 返回的是平方 L2 距离，越小越好；BM25 那一路（jieba 分词，k1=1.5、b=0.75）是无上界的绝对分，越大越好，且分值随查询长度与语料规模漂移。直接加权，等于让其中一路单方面决定排序。

处理分两步。第一步把距离换回相似度：向量已归一化，`cos = 1 - d²/2`，截断到 [0,1]，两路于是都变成「越大越相关」的同量纲分数。第二步处理 BM25 的无上界，用 `bm25_abs_normalize(beta=4.0, scale=4.0, raw_threshold=0.5)` 做 sigmoid 压缩，而不用 min-max。

选 sigmoid 的理由是可比的粒度。min-max 把本次检索的最高分强行映射为 1、最低分映射为 0，于是「整批都很差」与「有一篇很好」在数值上完全一样，下游无法区分「确实找到了」和「只能矬子里拔将军」。而且映射随每次查询变化，同一个块在两个查询下的分数不可比，α 这种固定权重就失去了意义。带绝对锚点的 sigmoid 保留了「原始分 4.0 附近算确有相关」这一先验：低于它趋近 0，高于它继续上升，分数因此跨查询可比。

α 取 0.20，依据是 37 题的扫描：

α	0.10	0.20	0.35	≥0.60
MRR	0.603	0.585	0.547	0.485

两点必须说明，否则这组数据会被读成「0.20 最优」。其一，0.10 的 MRR 其实高于 0.20，取 0.20 是在 0.10~0.40 这段差异落在噪声区间里的折中，不是挑峰值。其二，α ≥ 0.60 的劣化是真实的：0.485 相对 0.603 掉了一截，超出噪声。原因是语料全英文、提问全中文，BM25 能对上的主要是问题里夹带的英文标识符，权重一高就以字面重合压过了语义。

融合的收益有限但可测：hit@1 从纯向量的 15/37 提到 17/37，MRR 从 0.532 到 0.585，而「每题都取两路较优」的 oracle 上界是 0.642——现方案拿到这段互补红利的约 48%，另一半没拿到。候选池也不是瓶颈：BM25 召回条数 K 取 10、25、50、100，MRR 都是 0.550，没有变化，说明扩大召回补不上差距，问题在线性加权这个融合形式本身。代价是每次查询都要跑两路再做归一化，实测检索延迟均值 0.50 秒、最大 0.96 秒。

6.2 生成层：上下文归属与防编造

拼上下文时只带页码，是个看起来无害、实际会制造错误的选择。改动前每一块前面加 `[第%s 页]`。问题在于一次检索返回 10 个块，这 10 个块往往来自 6 篇文献，而「第 3 页」在六篇论文里都存在；模型拿到的文本流里，块与块之间没有任何标记说明这句话是谁说的。

因果链很短：归属信息缺失 → 模型无法判断当前这段文字属于哪一篇 → 十块不同来源的文本被揉成一段统一的「论文里的做法」→ 回答把甲文的设置安到乙文头上。这不是措辞问题，是可复现的事实错误。两个真实记录：把 CLIP 相关工作里提到的 Transformer 说成 Transformer 自己的做法；把《Scaling Laws》的 batch size（512 序列 × 1024 token）当成 Transformer 的超参数。

改法只有一行：每块前缀从页码换成「文件名 + 页码」（`[%s 第%s 页]`，取自 `metadata["source"]`）。同一份检索结果、同一个模型、同一套提示词，只改上下文格式，模型自己把两篇的设置分开了，并注明「这属于另一篇论文的设置」。单变量对照，改善可以归因到归属信息本身，而不是采样波动。

这处改动的价值不在「引用更好看」，而是同时压掉两类成本：事实错误，以及为防错不断加码的提示词。护栏最后只留两条：用户点名的文献完全没被召回时，必须单独写一行「（本次检索未返回该文献）」，不许拿别的文献顶替；若只是被别的论文顺带提到，要写明「仅《文件名》提到它，没有它自己的原文」。

护栏本身也有失败记录。第一版没有「部分命中」的边界，遇到「超参数是多少」这类问题时，因为召回里只有部分原文，模型直接整篇拒答，输出「参考资料未提及」——只回了 7 个字。补上的边界是：只要有部分原文，就先把有的写出来，缺的注明「资料中未给出这项」。防编造不能只做禁止，还要给「部分可用」留出口，否则会把可用信息一起丢掉。

缺口要承认：这处改动修的是归属，不是覆盖。问「Transformer 和 CLIP 有什么不同」时，若只召回一篇，模型只能给出一边，加文件名解决不了。

6.3 编排层：先看清单再检索

智能体在第一次检索之前，对「库里有什么」一无所知。它只知道用户问了什么，于是只能猜关键词、发一次检索、看返回、再猜。这种盲目试错一旦撞上工具调用次数上限，就变成错误结论：查了两三次没命中，模型回答「库中没有这篇」。

对照实验的题目是六篇全扫：「这几篇论文分别用了什么方法？各自的创新点是什么？」

配置	工具调用	总耗时	覆盖	结果
无上限	19 次检索	279.46 s	6 篇	内容全，但过渡语混进答案，且谎称联网查到
上限 6，无清单	10 次检索	83.93 s	2 篇	3 篇误报「库中没有」，实际在库
上限 6 + list_documents	1 次列目录 + 6 次检索	125.54 s	6 篇	六篇分节，标题全对

第二行是出发点：问题不是「少查了几次」，而是产生了三次假否定。假否定比漏答更糟——用户会据此认为自己的库不完整，转去重新上传或干脆放弃。

`list_documents` 刻意做得很轻：读 `chroma_structured.json`，返回「文件名 + 标题 + 页数」，不调模型、不走向量检索，实测 0.05 秒返回。它把「库里有什么」从「靠检索失败反推」变成一次显式查询，检索因此从盲目试错转为定向检索：先知道有哪六篇，再决定查哪篇、按什么顺序查。第三行里「六篇分节、标题全对」是直接结果——有了清单，模型才谈得上按文献组织答案。

代价要算清。125.54 秒约为无上限那次（279.46 秒）的 45%，覆盖率没降；比「上限 6 无清单」的 83.93 秒多 41.61 秒，换来覆盖从 2 篇到 6 篇。这笔交易是否划算取决于任务：单篇问答用不上清单，跨文献综述则必须用。

两处保留意见：检索次数上限最初只是提示词里的软约束，模型曾在一轮里搜了 22 次，现已改为代码强制（见 4.3.9）；清单工具的价值与库的规模相关，这里只有 6 篇，清单很短，库到上百篇时清单本身会挤占上下文，该规模下的表现未统计。

6.4 成本层：把「思考」当成可开关的开销

思考模式默认开着，很少有人把它当成一项可以单独计价的成本。同一份检索结果、同样 8888 字输入上下文，三种配置实测：

配置	耗时	输出 token	其中思考	正文
flash 默认（开思考）	82.27 s	4410	4103	678 字
flash 关思考	**8.77 s**	**464**	**0**	**948 字**
max 默认（开思考）	41.81 s	1610	1604	7 字

三行各说一件事。

其一，开思考时 4103/4410 即约 93% 的输出 token 花在思考上，正文只占一成。账单和延迟的绝大部分买的是看不见的过程。

其二，关掉思考后耗时从 82.27 秒降到 8.77 秒，约 9.4 倍，正文反而从 678 字涨到 948 字。这条最能说明「想得久不等于答得好」：在这个任务上，思考既拖慢了速度，又挤掉了答案的篇幅。

其三，max 那行是反例的极端形态：41.81 秒、1604 个思考 token，正文只有 7 个字——「参考资料未提及」。这 7 个字同时是 6.2 里护栏缺少边界的证据。成本付出去了，可用信息一个字没留下。

据此做了两个决定：模型从 qwen3.8-max 换成同代低一档的 qwen3.8-flash，请求里显式关闭思考（`extra_body={"enable_thinking": False}`）。方向是「降档 + 关思考」，不是「换更强的模型」。端到端效果：同一次问答从 79.29 秒降到 32 秒（本地），云端同一问题 34 秒，翻译一次 19 秒。

成本控制还有第二层：调用本身要有上限。原实现没有超时且 `max_retries=2`，一次卡住会被放大成 40×3=120 秒，一次问答调两三次模型就是三分钟以上；改为 `timeout=45, max_retries=0` 后，慢请求不再被重试放大。两处合起来才完整：单次成本要能关，失败成本要能止损。

空白也要标注：当前是全局关闭思考，没有按任务类型做消融。事实型问答的收益有上面三行支撑，但跨文献对比、多步推理这类任务关掉后是否掉质量，未统计。这也是把它做成「开关」而不是直接删掉的原因。

6.5 应用创新点

技术层回答「怎么让它不编」，应用层回答「它替用户省掉了哪一步」。四条都不是新能力，而是把已有能力按科研阅读的实际动线重新组织，取舍标准是**出错时代价最小的那个方向**。

应用创新	具体做法	用户省掉的步骤
可审计的答案	来源行由检索层从块元数据算出，不由模型自述；每块带「文件名 + 页码」	不必相信模型，直接翻到那一页核对
按节翻译而非整篇	以论文章节为单次翻译单元，中英对照，表格与公式原样保留，译文按段缓存	不必等一次长翻译，只翻要读的那一节

应用创新	具体做法	用户省掉的步骤
打开即问、上传幂等	六篇论文的结构化结果与向量库随仓库分发；上传按 md5 指纹判重，重复上传直接跳过	不必先准备语料，也不会重复入库
保守的失败姿态	查不到就答「库中没有相关资料」；点名文献未被召回时单列声明；禁止用通用知识补细节	不必替它分辨哪句是编的

第四条最容易被误读成缺点：一个什么都说不知道的助手看着不好用。依据是这个场景的错误代价——科研阅读里一条编造的数字要花半小时核对才能发现，而一句「没查到」只要换个问法再问一次。宁可少答，不可错答。

这四条各自的边界也要写明：可审计依赖解析质量，首页标题被切碎时页码对得上、原文却读不通；按节翻译依赖章节识别，标题层级乱掉的论文只能退化成整篇翻译；开箱即用是演示用的六篇快照，用户自己的论文仍需上传；保守拒答在跨文献对比问题上会显得过度谨慎，这正是 7.1 的短板。

6.6 与同类方案的差异

三类常见方案各自解决不同的问题，逐条说边界。

直接喂全文给长上下文模型。 六篇论文共 183 页、727 个块，最长一篇 48 页。窗口足够时，这条路省掉切块、嵌入、检索三整套工程，也不存在「召回不到」的问题——6.2 结尾那个「只召回一篇就只能答一边」的缺口，在它这里不会发生。它的代价在重复成本与可核对性：每个问题都要重新付全文的输入成本，回答里的页码也只能靠模型自己说。本系统选的是「一次入库、多次低成本检索」，代价是引入了召回失败这一新的失败模式。单篇精读、只问几个问题时，前者总成本更低，这一点不必回避；全文输入的实测成本对比未统计。

纯向量检索 + 通用 RAG 框架。 这类框架开箱即用，默认融合多是 RRF 或 min-max，默认的上下文拼接通常只有文本。本系统在两处做了不同选择（绝对锚点归一化、上下文带文件名），收益已在 6.1、6.2 量化。但不能据此说这套做法更优：融合只拿到 oracle 上界的 48%，而 RRF 只比名次、对候选池不敏感，恰是 5.4 指出的可能突破方向。本系统没有做 RRF 的对照实验，未统计。这里胜出的不是「更好的融合算法」，而是「针对中文提问 + 英文语料这一组合调过的权重」。

商用 PDF 阅读器 / AI 阅读工具。 这类产品在解析鲁棒性和单篇阅读体验上明显更强，版面还原、图表定位、断字处理都经过大量真实文档打磨。本系统的解析层有明确短板：表格块 markdown 化后与「超参数」这类字面提问重合度低，有时要第二次检索才命中；第 1 页的粗体标题常被切成三段，界面上显示为「Lo w-Rank」。所以「读一篇论文」这件事，商用工具是更合适的选择。本系统的取舍在另一侧：本地库、可离线部署、检索过程可见（每块带文件名与页码）、跨文献对比、按章节翻译，以及一份能自己改的代码。

方案	更适合的场景	本系统在此处的取舍
全文喂长上下文	单篇精读、问题少、窗口够	换成一次入库多次检索，接受召回失败风险
纯向量 + 通用 RAG	快速搭原型、语料单一	调过权重的线性融合，未与 RRF 对照
商用 PDF 阅读器	单篇阅读体验、复杂版面	放弃解析鲁棒性，换本地化与可核对

四处改动可以收成一句：每一处都对应一条具体的失败记录，而不是一条功能清单。sigmoid 对应「整批都差」无法识别，文件名对应两条归属错误，清单工具对应三次假否定，关思考对应 41.81 秒换 7 个字。拿不出这种对应关系的地方都标了未统计——RRF 对照、清单在上百篇规模下的开销、按任务类型的思考消融、全文输入的实测成本。这些是当前说不清的部分，也是下一步要补的实验。

第 7 章 总结与展望

本章列的是交付前自查中已经复现过的六条缺陷，不是“可能存在”的风险清单。每条按四段写：**现象**（用户看到什么）、**影响**（坏在哪）、**目前怎么绕**（现在靠什么撑着）、**根因**（为什么提示词或调参解决不了）。用到的数字全部取自同一份事实卡；没有实测的部分写“未统计”，不估。

先说结论：这套系统的短板不在生成，而在两处更靠前的地方——**检索覆盖**与**解析质量**。模型能做的只是把拿到的块组织成答案，拿不到的东西它编不出来，也不该编。

7.1 检索层面

7.1.1 表格块与字面提问的重合度低

现象：727 个块里有 82 个表格块、645 个正文块。PDF 转成 markdown 后，表格变成由竖线和短词组成的稀疏文本，表头与提问词之间几乎没有连续语义。问「attention is all you need 中，超参数是多少」，第一轮检索经常召回正文里讨论实验设置的段落，而不是 Table 3；要第二轮换个说法才命中。同一问题端到端 32 秒（云端 34 秒），耗时里就包含不止一次检索与生成。

影响：单轮问答答不全，或者答成另一篇论文的设置——结构化抽取统计到 102 张表，进入向量库的表格块只有 82 个，两个口径本来就不对齐，混在一起排更容易串。用户看到的是一段语气笃定、但数值取自别处的回答。

目前怎么绕：靠智能体多轮检索补第二次、第三次；靠上下文带文件名（`[文件名 第 N 页]`）把跨论文的归属错误压下去；靠提示词要求在缺项时写“资料中未给出这项”，而不是整篇拒答。表格本身没有任何特殊处理。

根因：表格与正文走同一条切分链路（`chunk_size=1500, chunk_overlap=150`），表格的行列结构在 markdown 化时已经丢了，向量化只看到“词袋式的表格碎片”。这是入库策略问题，改提示词不会让 Table 3 变得更容易被召回。

7.1.2 跨文档对比完全取决于检索覆盖

现象：问「Transformer 和 CLIP 有什么不同」，如果 Top-10 只召回了其中一篇的块，模型就只给得出一边。另一边要么整段缺失，要么拿别的论文里顺带提到的一句话顶上（后者已被提示词护栏拦住，要求单独写「本次检索未返回该文献」）。同一个现象在六篇全扫的题上更明显：「这几篇论文分别用了什么方法？各自的创新点是什么？」在上限 6 次、且没有清单工具时，只覆盖 2 篇，另有 3 篇被误报“库中没有”——它们明明在库里；加上 `list_documents` 后，1 次列目录加 6 次检索就覆盖了 6 篇。

影响：这是最容易被忽略的一类错误。它不是答错，是答漏，而漏掉的一半在版式上看不出来——两段并列、标题齐全，读者没有理由怀疑少了一篇。

目前怎么绕：`list_documents` 先取“文件名 + 标题 + 页数”清单（读 `chroma_structured.json`，不调模型、不检索，0.05 秒返回），再按清单逐篇检索；提示词要求点名文献缺失时单独写一行，只被别的论文提到时写「仅《文件名》提到它」。

根因：融合是全局排序，`final = 0.20 · BM25_norm + 0.80 · cos`，取 Top-10。没有“每篇至少一块”的约束，一篇论文的相关块多就会把另一篇整体挤出去。调参也救不了：候选池 K 从 10 扩到 100，MRR 一直是 0.550，说明瓶颈不在池子大小；α 从 0.20 调到 0.10 也只到 0.603，仍低于 oracle 的 0.642。**这是配额问题，不是分数问题，提示词改不了。**

7.2 解析层面

7.2.1 首页标题被字距加粗拆成单字母

现象：部分论文用字距加粗做主标题，`pymupdf4llm` 按字距判断加粗边界，解析结果是 `**Lo** w- **R**ank`，界面上显示为「Lo w-Rank」。

影响：三处。标题展示错误，用户第一眼就看见；按标题做的字面匹配失效，BM25 通道拿 `Lo`、`w-`、`R` 去撞“LoRA”“Low-Rank”，撞不上；按章节翻译时译文会带着拆开的字母。语义召回还能兜住大部分问题，这个错误不会让整页失效，只是看起来不专业。

目前怎么绕：没有专门处理。属于已知偏差，交付版保留原样，不在展示层做字符串修补——修补只能骗过眼睛，修补不了向量里已经存下的错误文本。

根因：解析层把排版信号（字距）误当成语义信号（加粗）。修它要重跑受影响块的切分与嵌入，代价与收益不匹配，所以排在交付之后。

未统计：受影响的块数与页数。

7.3 工程层面

7.3.1 六份存储非原子写入

现象：一次入库要写六处——`chroma/`、`chroma_summary.json`、`chroma_structured.json`、`bm25.pkl`、`md5.txt`、`embedding.json`，顺序固定为 chroma → summary → structured → bm25 → md5 → embedding.json。云端实例重启若正好落在中间，会出现部分回滚：向量库有块、结构化文件没记录，或者反过来。线上表现为“文件显示已入库但不报相似”、“有 7 个文件却检索不到”。

影响：一致性靠运气。用户以为入库成功，实际检索不到；更糟的是它会一直留着，直到下次入库覆盖或人工清理。

目前怎么绕：启动时对账。以 chroma 的实际块为基准，和其余五份的键做差集，只做两件事——单侧空壳删记录，单侧内容补记录；不重嵌入、不重建索引。

根因：架构级，没有事务、没有两阶段提交、也没有单一事实源，六个文件各自是“真相”。对账只能把状态收敛到一致，保证不了写入过程不出现中间态。**短期只能缓解，不能根治**；根治要把六份合成一份带版本的存储，或者把写入改成“先写临时目录、成功后原子替换”。

7.3.2 检索次数上限是提示词软约束

现象：提示词里写“最多检索 6 次”，实测同一道题实际调了 10 次检索、83.93 秒。无上限那组更极端：19 次检索、279.46 秒，过渡语混进最终答案，还出现“谎称联网查到”。

影响：延迟不可预期，用户等多久没有上限；token 成本随轮次线性涨；检索太多还会把上下文塞满，挤掉真正关键的块。

目前怎么绕：靠提示词写规则、靠界面显示“共检索 N 次，用时 X 秒”让用户看见实际发生了什么、靠把每轮过渡语移到上方灰色小字，不拼进答案。没有硬中断。

根因：上限写在系统提示词里，模型可以遵守也可以不遵守，执行层没有计数器强停，也没有“预算耗尽就收口”的兜底分支。这和 7.1.2 是同一类问题：**软约束在边界模糊时必然失准**，只有落到代码里的约束才算数。

7.3.3 联网开关依赖外部密钥

现象：未配 `TAVILY_API_KEY` 时，`web_search` 只返回一句提示，但开关仍在界面上，仍可被打开。

影响：用户以为能联网，拿到的是一句提示语；如果模型把这句提示当成检索结果继续推理，就会复现 7.3.2 里“谎称联网查到”的情形。

目前怎么绕：`pick_prompt` 这个 `@dynamic_prompt` 中间件读 `request.runtime.context["use_web"]`，决定是否追加联网规则；工具本身返回明确提示，不假装有结果。

根因：能力开关与部署态配置没有联动，属于配置问题而非代码缺陷，但用户体验上它是一个“看起来能用、其实不能用”的开关。

7.4 评测层面

现有评测集 37 题，只测检索，不测答案。它足以支撑第 5 章的融合结论，但不足以发现本章的六条缺陷，原因有四条。

一、hit@10 已经饱和。 纯向量 hit@10 是 33/37，融合也是 33/37，两者打平；能区分的只有 hit@1（15/37 → 17/37）和 MRR（0.532 → 0.585）。也就是说，任何“让第 6 到第 10 位变好”的改动，在这个指标上都看不出来。

二、题型不覆盖。 37 题是事实查找型，没有跨文档对比题、没有表格题、没有“这张图说明了什么”。7.1.1 和 7.1.2 两类失败恰好都落在题型之外，所以“评测通过”并不等于“这些缺陷不存在”。各题型的具体数量未统计。

三、判定口径偏松。 命中只比对块所在页与标准答案页是否一致，不校验该块是否真的覆盖了答案句。同一结论在引言和结论各说一遍时，召回后者算命中；落在该页却没覆盖答案的块也算命中。hit@10 因此偏宽松，MRR 相对严格。

四、单次运行、无重复。 0.02 量级的差异在 37 题上不可分辨；K 取 10、25、50、100 时 MRR 全是 0.550，也说明这个题集对参数不敏感。所以第 5 章只主张“$\alpha \geq 0.60$ 明显变差”这类量级判断，不主张某个 α 唯一最优。

缺的那一维是**答案质量**：引用是否真的支撑结论、有没有越界推断、页码是否可信，目前没有任何自动检查，全靠提示词自觉。

7.5 后续改进

四条近期改进逐条展开，其余方向统一进 7.5.5 的总表；所有条目都未在本次交付中实施。

7.5.1 按文献分组的平衡检索（优先级 1）

做法：融合排序之后、拼上下文之前，按 `source` 分组，先给每篇保底 1 块——6 篇最多占 6 席，剩下 4 席仍按 `final` 分数填。某篇一块都没有时不硬凑，但在结果里记「未覆盖」，由提示词转成「本次检索未返回该文献」。

改哪里：`rag.py` 拼 `context` 之前的一段排序，不动索引、不动嵌入、不重跑入库。

验证与代价：用「Transformer 和 CLIP 有什么不同」和六篇全扫那题各跑几遍，看覆盖篇数是否稳定到 6、MRR 是否掉；中低代价，风险是保底配额挤掉高分块。

7.5.2 启动时对账（优先级 2）

做法：进程启动时读 chroma 的 id 集合，与 `chroma_structured.json`、`md5.txt`、`embedding.json` 的键求差集；只有记录没有块的删记录，只有块没有记录的补记录，不重嵌入、不重建 `bm25.pkl`。

改哪里：`knolege.py` 的初始化路径加一个幂等的 `reconcile()`，返回并打印差集条数。

验证与代价：写入中途 kill 进程，重启后看六份是否收敛，再连续入库两次确认不误删；低代价，但它只是把不一致收敛掉，7.3.1 的非原子写入根因还在。

7.5.3 思考模式做成用户可见开关（优先级 3）

做法：把 `extra_body={"enable_thinking": ...}` 从写死改成侧栏开关，默认关。

依据：同一次检索、8888 字上下文下，flash 开思考 82.27 秒 / 4410 token（思考占 4103）、正文 678 字；关思考 8.77 秒 / 464 token、正文 948 字。

验证与代价：同组题在开关两态下比耗时与正文长度；低代价，参数已存在，只加 UI 与透传，难在向用户解释什么时候该开。

7.5.4 表格单独入库（优先级 4）

做法：入库时把 82 个表格块分流，单独 collection 或至少独立 metadata，配独立切分策略——保留表头，并给每一行拼「表名 + 列名」，让一行数据自带语义。

改哪里：入库流水线第 2 至 4 步加一条按块类型分流的支路；`rag.py` 召回侧加一个按查询词触发的通道选择。

验证与代价：以「attention is all you need 中，超参数是多少」看是否首轮命中 Table 3；中高代价，需重嵌入 82 个表格块并回归 37 题，正文块不用动。

7.5.5 优先级总表

上表四条是交付后最先该做的。下表把近期四条与其余长期方向放在同一把尺子上排：收益看它解决了哪个已复现的问题，代价看改动面。

优先级	事项	预期收益	代价
1	按文献分组的平衡检索（7.5.1）	跨文档对比不再只答一边，覆盖可预期	中低
2	启动时对账（7.5.2）	消除“入库成功但检索不到”	低
3	思考模式做成开关（7.5.3）	速度与质量可切换，延迟可控	低
4	RRF 替换线性加权	拿回 0.585 到 oracle 0.642 之间的一部分差距	低（半天）
5	4 道失败题逐块排查	定位附录与表格区的召回失效原因	低（仅分析）
6	表格单独入库（7.5.4）	表格题首轮命中，少一轮检索	中高
7	图注配对 + 通用 VLM 描述入库	补上 138 张图的能力缺口，能回答“图说明了什么”	中
8	BM25 索引改为启动构建 + 增量更新	检索延迟明显下降	中（与 13 同设计）
9	智能体职责分化与统一调配	把软约束变成结构约束，调度准确率可单独评测	中高
10	回答评判机制	答案质量可事后复查，评测多一个维度	中高
11	硬连字符还原 + 按小节对齐切点	块结尾完整率提升	高（需全部重嵌入）
12	公式识别	补上公式抽取	高（新增能力）
13	FastAPI 前后端分离	消除重复解析、解除单进程约束、接口可复用	高
14	LoRA 微调小模型做图表描述	图表描述的质量与成本双改善	高（需 GPU 与标注）
15	自进化闭环	评测集随使用增长，参数可持续校准	高（依赖 10）

整体排序看的是收益与代价之比。第 1 至 5 项都是「改一处、当天能验证」的量级：第 1 项补 7.1.2 的跨文档失败；第 4 项去拿 0.642 与 0.585 之间那 0.057，换成 RRF 这类只看名次的融合能吃回一部分；第 5 项不改代码，先把 4 道失败题逐块拆开，结论直接决定第 6、11 两项要不要做。

第 6 项之后改动面变大：第 7、12 两项是补能力缺口，第 13 项是架构级改造且必须与第 8 项一起设计，第 11 项代价最高——全部块要重嵌入而收益只是结尾完整率，除非第 5 项证明断块是主要漏召回原因，否则不该先动；第 14、15 项是研究方向，不适合在交付前动。

7.6 总结

这套系统擅长「有明确出处的事实查找」：6 篇论文、727 个块，答案带文件名与页码，可逐条核对。不擅长三类问题：跨文献对比时检索只覆盖一篇、答案藏在表格或图里、首页这类解析受损的位置。边界不在生成，而在检索覆盖与解析质量。